第3章 进程管理

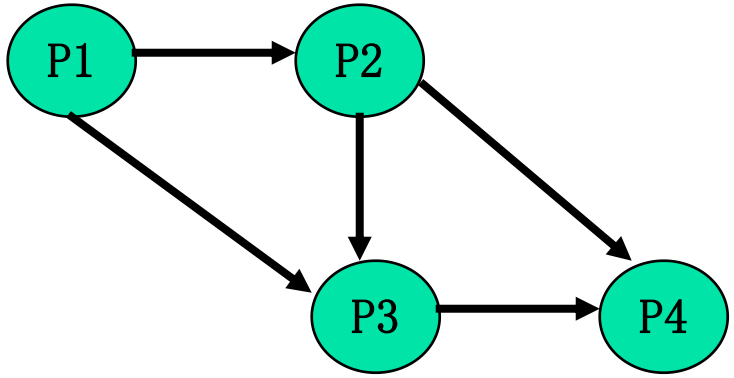
3.1 进程的基本概念
3.2 进程的描述
3.3 进程控制
3.4 进程互斥与同步
3.5 进程通信
3.6 线程

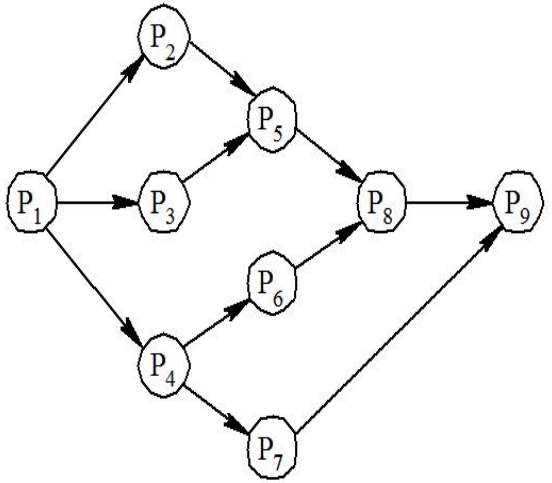
3.1.1 前趋图

- 前趋图（Precedence Graph）
  - 前趋图是一种用来描述程序或程序的程序段之间先后执行顺序的有向无环图。
  - 图中每个结点可以表示一条语句、一个程序段、一个程序或进程。
  - 若两个结点P_i、P_j，仅当P_i操作完成后，才能执行P_j结点的操作，称P_i，P_j之间存在前趋关系，表示为：$Pi \rightarrow P_j$
  - 前趋关系是一个偏序关系。

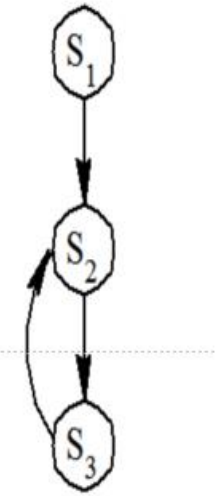
3.1.1 前趋图

- 前趋图（Precedence Graph）





- 前趋图中不允许有循环，否则会产生不可能实现的前趋关系

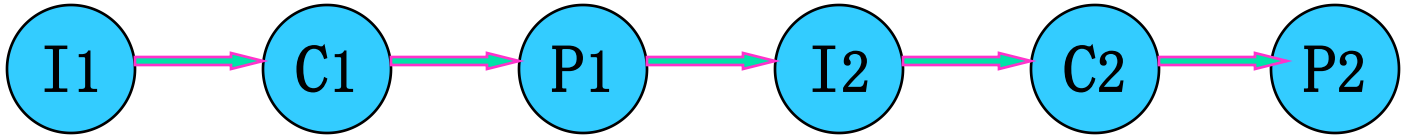


3.1.2 程序的两种执行方式

- 程序执行的两种方式
  - 顺序执行
  - 并发执行

3.1.2 程序的两种执行方式

- 程序的顺序执行包含两层含义：
  - 对于多个用户程序来说，所有程序是依次执行的。（外部顺序性）



  - 对于一个程序来说，它的所有指令是按序执行的。（内部顺序性）

```
S1:   a：=x+y;
S2:   b：=a-5;
S3:   c：=b+1;
```

3.1.2 程序的两种执行方式

- 程序顺序执行的特征
  - 顺序性：指令严格处理机的操作严格按照程序所规定的代码书写顺序一条一条地执行。
  - 封闭性：程序一旦开始执行，就独占熊所有地做元（CPU，内存，硬件设备等），其执行状态只受程序指令自身地影响，外界因素无法干扰或改变它的运行。
  - 可再现性：只要程序执行时的环境和初始条件相同，当程序重复执行时，无论快慢，都将获得相同的结果。

3.1.2 程序的两种执行方式

- 程序的并发执行就是一组在逻辑上互相独立的程序或程序段在执行过程中，其执行时间在客观上互相重叠，即一个程序段的执行尚未结束，另一个程序段的执行已经开始的这种执行方式。
- 并发执行可能是：
  - 多道程序的并发执行
  - 某道程序的几个程度段中包含着一部分可以并发执行的代码

3.1.2 程序的两种执行方式

- 问题：程序并发执行会发生什么？
- 举例：
  - P1：（1）M=account；（2）M=M+100；（3）account=M；
  - P2：（4）N=account；（5）N=N+100；（6）account=N；

    P1和P2都执行完毕，可能出现几种运行结果？

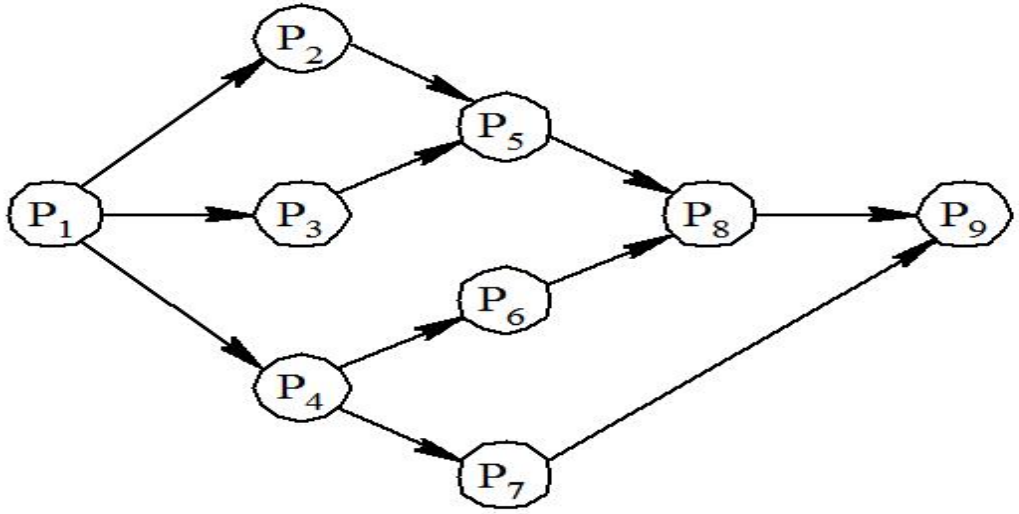
- 总结：
  - 同样的程序，并发执行的过程不同，得到的结果可能不同。说明并发执行环境下，程序的并发执行是间断执行，失去了封闭性和可再现性。
  - 程序已不能完全代表程序的执行过程！

3.1.2 程序的两种执行方式

- 程序并发执行的特征
  - 间断性:进程间可能存在的制约关系导致并发程序执行具有“执行–暂停–执行”的间断性执行的表现。
  - 失去封闭性：系统中存在多个执行的程序时，系统资源为多个并发程序共享。一个程序运行时，其执行环境受到其他程序执行的影响，如共享CPU
  - 不可再现性：因为失去了封闭性，程序执行结果与程序执行的速度有关，程序经过多次执行后，虽然它们执行的环境和初始条件相同，得到的结果可能却各部相同。

3.1.2 程序的两种执行方式

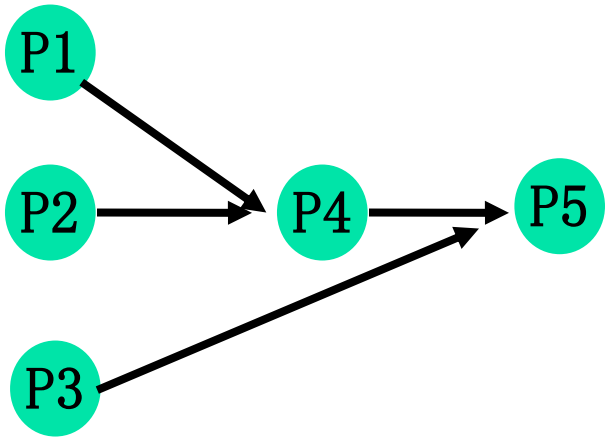
- 注意：只有再逻辑上互相独立的程序或程序段才能并发执行。具有前趋关系的程序段或程序间是不能并发执行的。



- 如图2，P2、P3、P4可以并发执行，P1和P2不能并发执行。

3.1.2 程序的两种执行方式

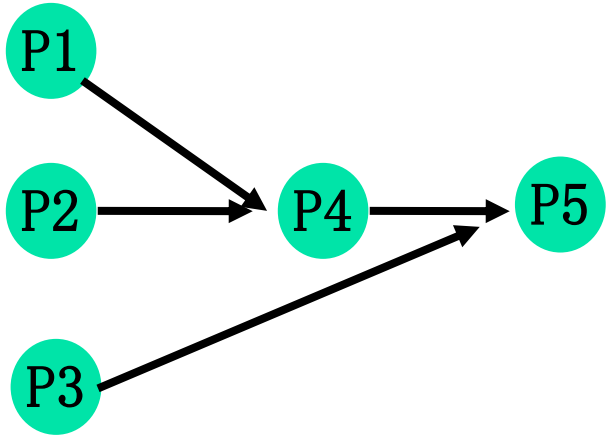
- 例:表达式　　(a+b)*(c+d)-e/f
  - 设：P1=a+b，P2=c+d，P3=e/f，P4=P1*P2，P5=P4-P3，给出前趋图，哪些可以并发执行?



3.1.2 程序的两种执行方式

- 程序并发执行的伪码表示

```
Parbegin
  S1;S2;...;Sn;
Parend;
```




```
Begin
   parbegin
      begin
         parbegin
            P1;P2
         parend
        P4;
      end;
        P3;
   parend
    P5;
end;
```

3.1.3 进程的基本概念

- 思考：

  OS分配资源面向的对象是程序还是程序执行的过程？

- 由前面的讨论知：

  程序的并发执行下，程序不能完全代表程序的执行过程

3.1.3 进程的基本概念

- 进程（Process）
  - 进程是程序在一个数据集上的运行过程，是系统进行资源分配和调度的一个独立单位。
- 其它定义：
  - 进程是程序的一次执行。
  - 进程是一个程序及其数据在处理机上顺序执行时所发生的活动。
  - 进程是进程实体的运行过程，是系统进行资源分配和调度的一个独立单位。

3.1.3 进程的基本概念

- **进程的特征**
  - **动态性**：是一次运行过程，有生命周期；
  - **并发性**：可并发执行；
  - **独立性**：作为一个独立单位运行；
  - **制约性**：进程间由于对资源的争用而相互制约；
  - **异步性**：进程按照各自不可预知的速度向前推进；
  - **结构特征**：进程=程序+数据+PCB；

3.1.3 进程的基本概念

- 进程和程序的关系和区别
  - 进程是程序的执行，是动态的概念，有生命周期；程序是一组指令的有序集合，是静态的永久存在；
  - 进程组成：进程=程序+数据+PCB；
  - 进程可并发执行；
  - 进程可作为一个独立的单位运行，同时也是系统中独立获得资源和独立调度的基本单位；凡未建立进程的程序，都不能作为一个独立的单位运行；
  - 一个程序可对应多个进程；
  - 一个进程可包含多个程序。主程序进程执行时可调用其它程序，共同组成一次“活动”；

3.2 进程的描述

- 进程控制块（PCB）
  - 从理论角度看，进程是对正在运行的程序过程的抽象。
  - 从信息系统内的实现角度看，OS需要创建一个专门的数据结构来描述进程，OS通过该数据结构描述进程的基本情况和活动过程，进而控制和管理进程，这个数据结构简称为进程控制快（Process Control Block，PCB）。

3.2 进程的描述

- 进程的描述
  - 系统中需要有描述进程存在和能够反映其变化的物理实体，即进程的静态描述。
  - 进程的静态描述由三部分组成：进程控制块PCB+有关程序段+该程序段对其进行操作的数据结构集。
  - 进程控制块PCB包含了有关进程的描述信息、控制信息以及资源信息，是进程动态特征的集中反映。
  - 系统根据PCB感知进程的存在，通过PCB中所包含的各项变量的变化，掌握进程所处的状态以达到控制进程活动的目的。
  - 进程的PCB是系统感知进程的唯一实体，常驻内存。

3.2 进程的描述

- 进程控制块
  - PCB中记录了操作系统所需的、用于描述进程当前情况以及控制进程运行的描述信息、控制信息和资源信息
  - 具体包括下述四方面的信息：

（1）进程标识符：内部标识符（进程号）；外部标识符（名）；父进程标识及子进程标识；用户标识；

（2）处理机状态：处理机状态信息主要由处理机的各种寄存器中的内容组成的。寄存器包括：通用寄存器、指令计数器、程序状态字（PSW）寄存器、栈指针等。

当处理机被中断时，这些信息都必须保存到PCB中，以便该进程重新执行时，能从断点继续执行。

3.2 进程的描述

- **进程控制块**

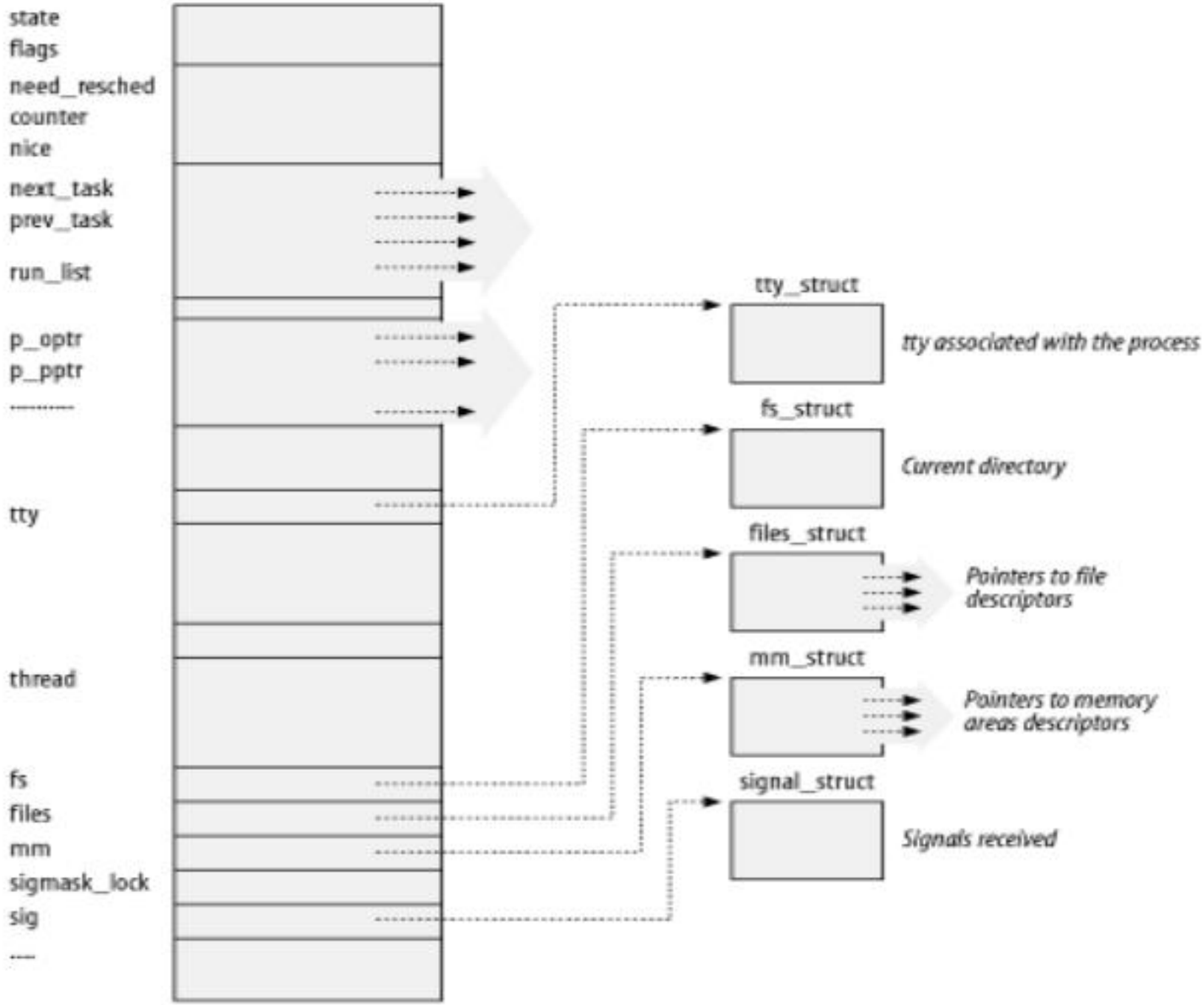
（3）**进程调度信息**：在PCB中还存放一些与进程调度和进程对换有关的信息。包括：进程状态 ，进程优先级，进程调度所需的其它信息，事件；

（4）**进程控制信息，包括**：

- 程序和数据的地址——指程序和数据所在的内存或外存首地址；
- 进程同步和通信机制——如信号量、消息队列指针等，它们可能全部或部分地存放在PCB中；
- 资源清单——是一张列出了除CPU外的、进程所需的全部资源及已经分配到该进程的资源的清单；
- 链接指针——它给出本进程（PCB）所在队列中下一个进程的PCB的首址。

3.2 进程的描述

- 进程控制块
  - Linux进程描述符由一个task_struct结构表示，task_struct结构是一个复杂的结构，占一千多字节，其各个成员用来准确描述进程在各方面的信息. 主要有以下几个部分：


state
flags
need_resched
counter
nice
next_task
prev_task
run_list
p_optr
p_pptr
tty
thread
fs
files
mm
sigmask_lock
sig
tty_struct
tty associated with the process
fs_struct
Current directory
files_struct
Pointers to file descriptors
mm_struct
Pointers to memory areas descriptors
signal_struct
Signals received

3.2 进程的描述

- PCB的组织
  - **链接方式**：把具有同一状态的PCB，用其中的链接字链接成一个队列。
    形成：就绪队列、阻塞队列、空白队列等。

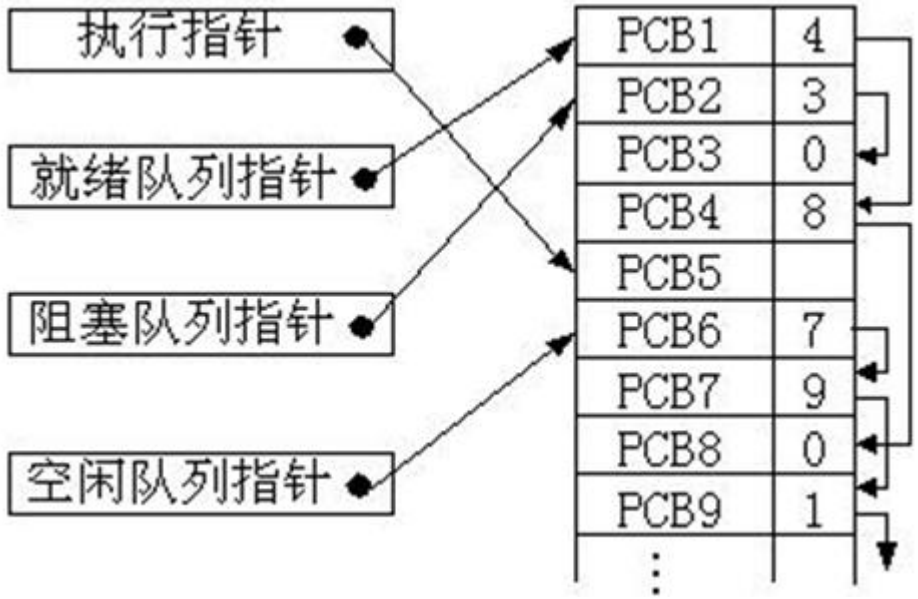


图3-2 PCB表的链接组织

3.2 进程的描述

- PCB的组织
  - **索引方式：** 系统根据所有进程的状态建立几张索引表。如就绪索引表、阻塞索引表等。索引表的首址记录在专用单元中；每个索引表的表目中，记录具有相应状态的某个PCB的首址。

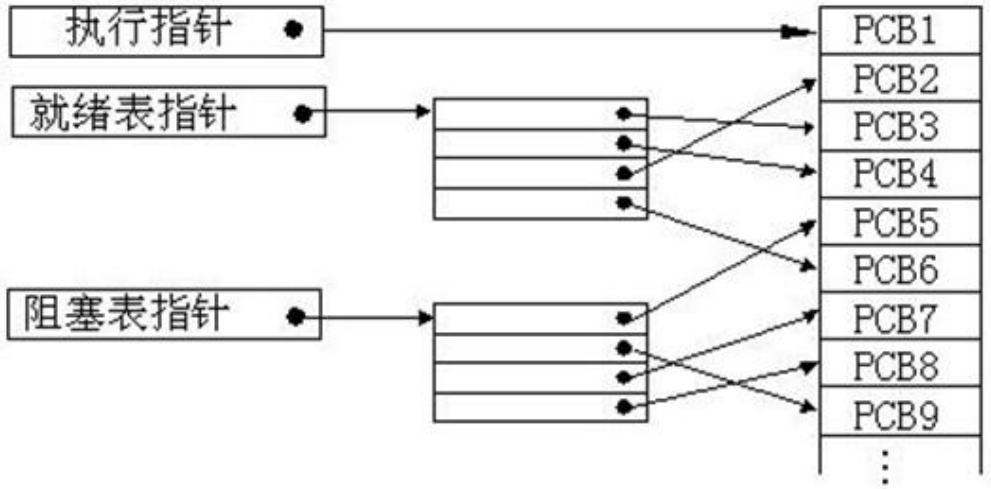


图3-3 PCB表的索引组织

3.3 进程控制

- 进程的基本状态
  - 就绪（Ready）状态：当进程已分配到除CPU以外的所有资源后，只要再获得CPU，便可立即执行，进程这时的状态称为就绪状态。
  - 执行（Running）状态：进程已获得CPU，其程序正在执行。
  - 阻塞（Blocked）状态:正在执行的进程由于发生某事件而暂时无法继续执行时，便放弃处理机而处于暂停状态，亦即进程的执行受到阻塞，把这种暂停状态称为阻塞状态（或等待状态）。

3.3 进程控制

- 进程的基本状态
  - 创建状态：当一个新进程被创建时，系统已经为其分配了PCB，填写了进程标识等信息，但是该进程所必须的资源或其他信息如主存资源等尚未分配。
  - 注意：此时的进程已经拥有了自己的PCB，但进程自身还未进入主存，即进程创建工作尚未完成，进程不能被调度执行，其所处的状态就是创建状态。

3.3 进程控制

- 进程的基本状态
  - 终止状态：当一个进程到达了自然结束点，或是出现了无法克服的错误，或是被操作系统所终结，或是被其他有终止权的进程所终结，它将进入终止状态。
  - 进入终止状态的进程以后不能执行，但在操作系统中依然保留一个记录，其中保存状态码和一些计时统计数据，供其他进程收集。一旦其他进程完成了对终止状态进程的信息提取之后，操作系统将删除该进程。

3.3 进程控制

- **进程的基本状态**
  - **进程的挂起状态：**挂起状态指的是进程的代码和数据资源暂时离开内存，被交换到外存上，以节省内存资源供其他进程使用。
  - **进程的解挂状态：**解挂状态是指将原本挂起的进程从外存（如磁盘）上重新载入内存，并恢复其执行或准备执行的状态。解挂后，进程可以重新参与系统的调度和执行。

3.3 进程控制

- 进程的基本状态
  - 进程的挂起/解挂状态：
    - 设置原因：
      - 用户需要：中间结果与预期不符；
      - 操作系统需要：系统某些功能故障；
      - 系统负荷过重
      - 父进程需要
      - 对换的需要
    - 设置挂起状态后，进程的就绪、阻塞状态分别分为：
      - 就绪：活动就绪　Readya　静止就绪　Readys
      - 阻塞：活动阻塞　Blockeda　静止阻塞　Blockeds

3.3 进程控制

- 进程的基本状态

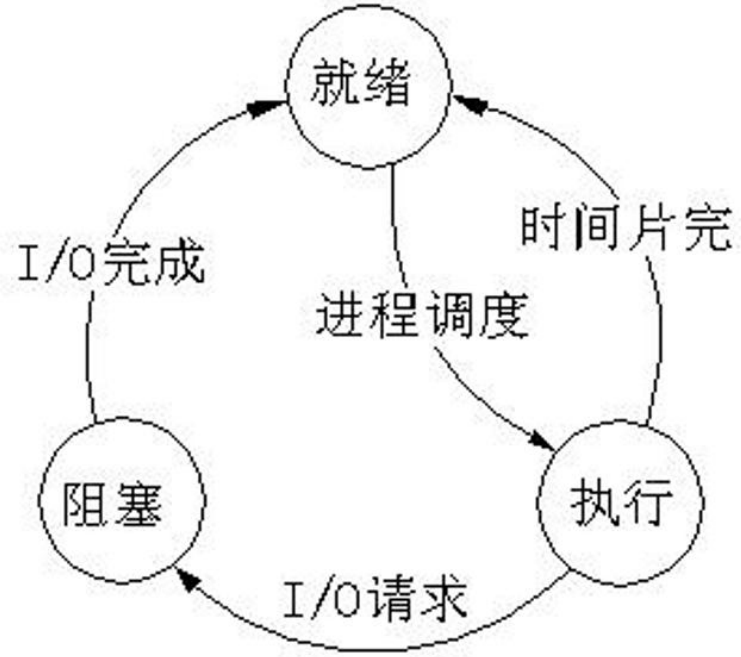


图3-4 进程的三种基本状态及其转换

3.3 进程控制

- 进程的基本状态

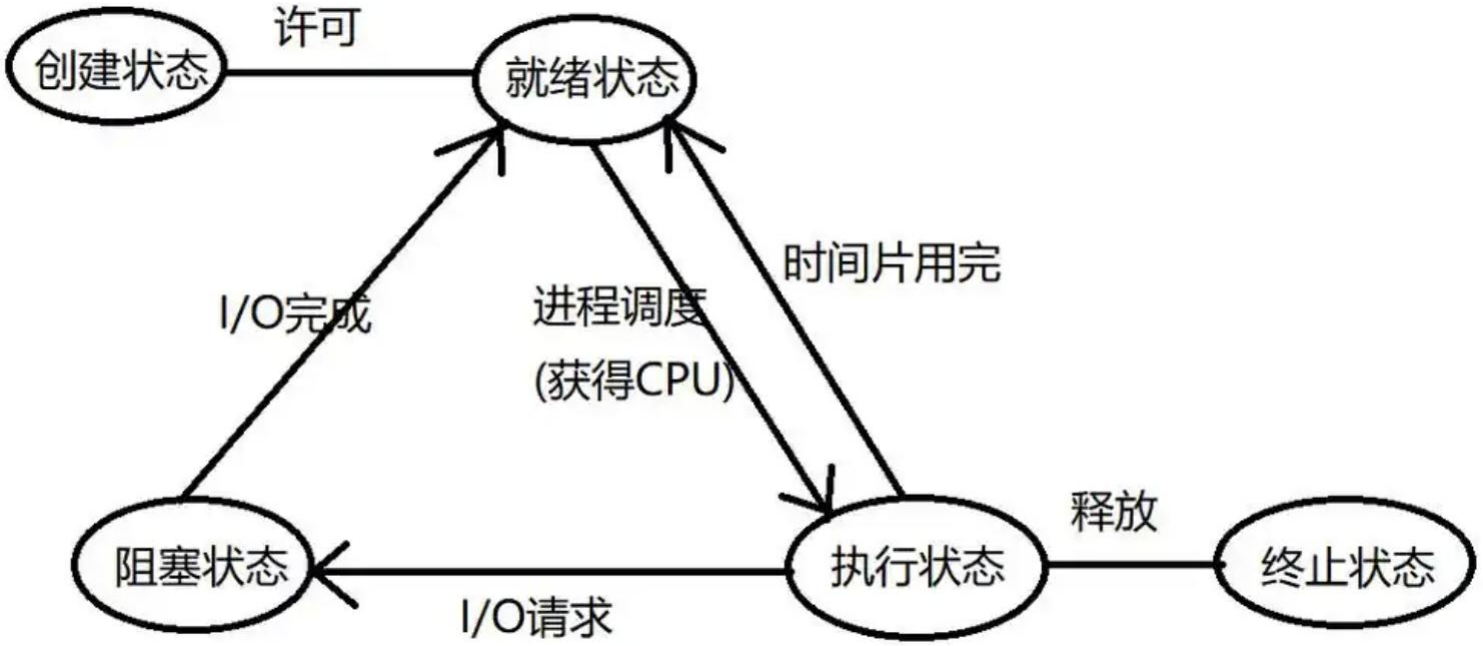


图3-5 具有创建、终止状态的进程状态图

3.3 进程控制

- 进程的基本状态

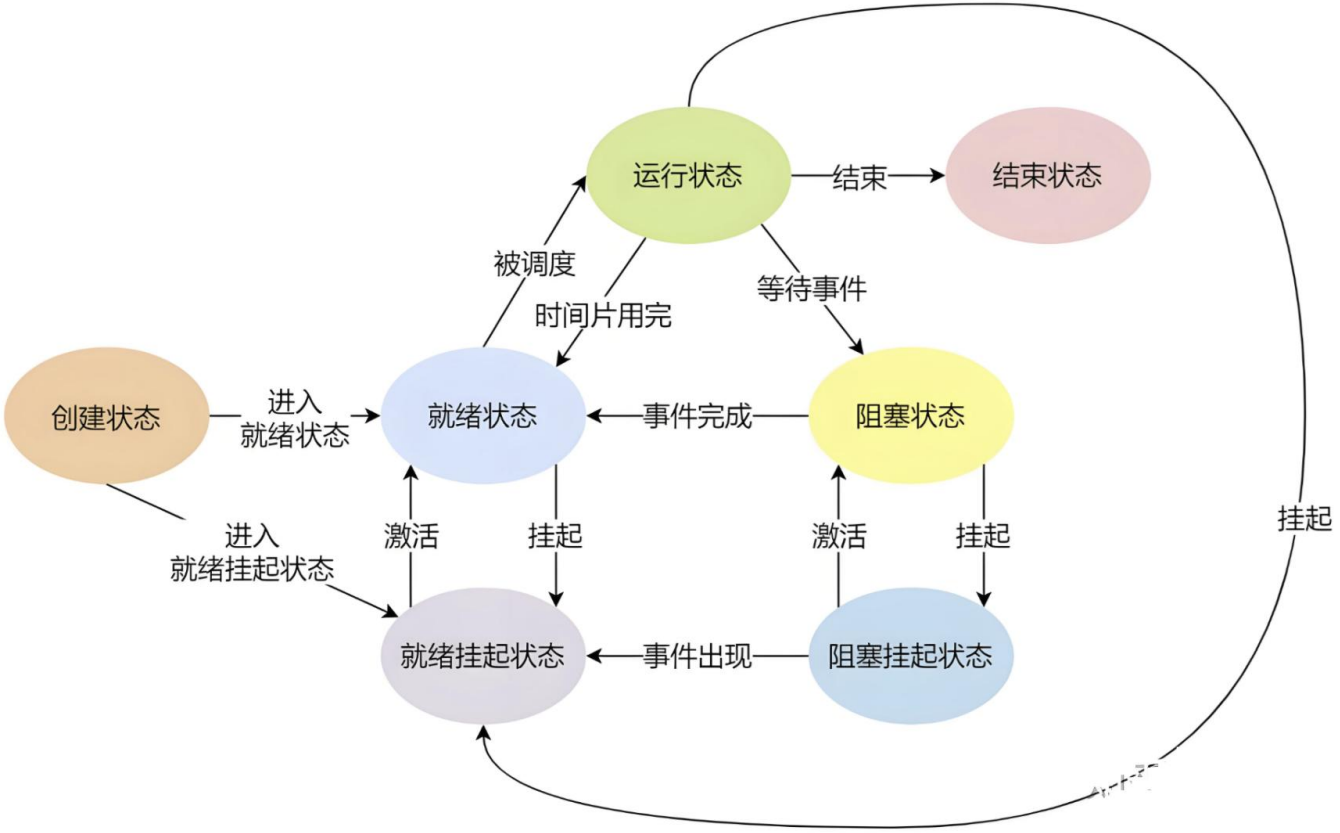


图3-6 具有创建、终止和挂起状态的进程状态图

3.3 进程控制

- 进程控制原语
  - 进程控制就是系统使用一些具有特定功能的程序段来创建、撤销进程以及完成进程各状态间的转换，从而达到多进程高效率地并发执行和协调，实现资源共享的目的。
  - 原语：把系统态下执行的某些具有特定功能的程序段称为原语。原语有指令级和功能级的（程序段）。这两类原语都在系统态下执行，完成某个系统管理所需要的功能和被高层软件调用。原语的执行具有原子性。
  - 在操作系统中，通常把进程控制用程序段做成原语。

3.3 进程控制

- 进程控制原语–进程创建
  - 引起创建进程的事件
    - 用户登录
    - 作业调度
    - 提供服务
    - 应用请求
  - 无论是哪一种方式创建进程，在系统生成时，都必须由操作系统创建一部分承担系统资源分配和管理工作的系统进程，通过调用创建原语实现。

3.3 进程控制

- 进程控制原语-进程创建Creat()
  - 创建原语的处理过程

（1）申请空白PCB；

（2）为新进程分配资源。主要是内存空间。

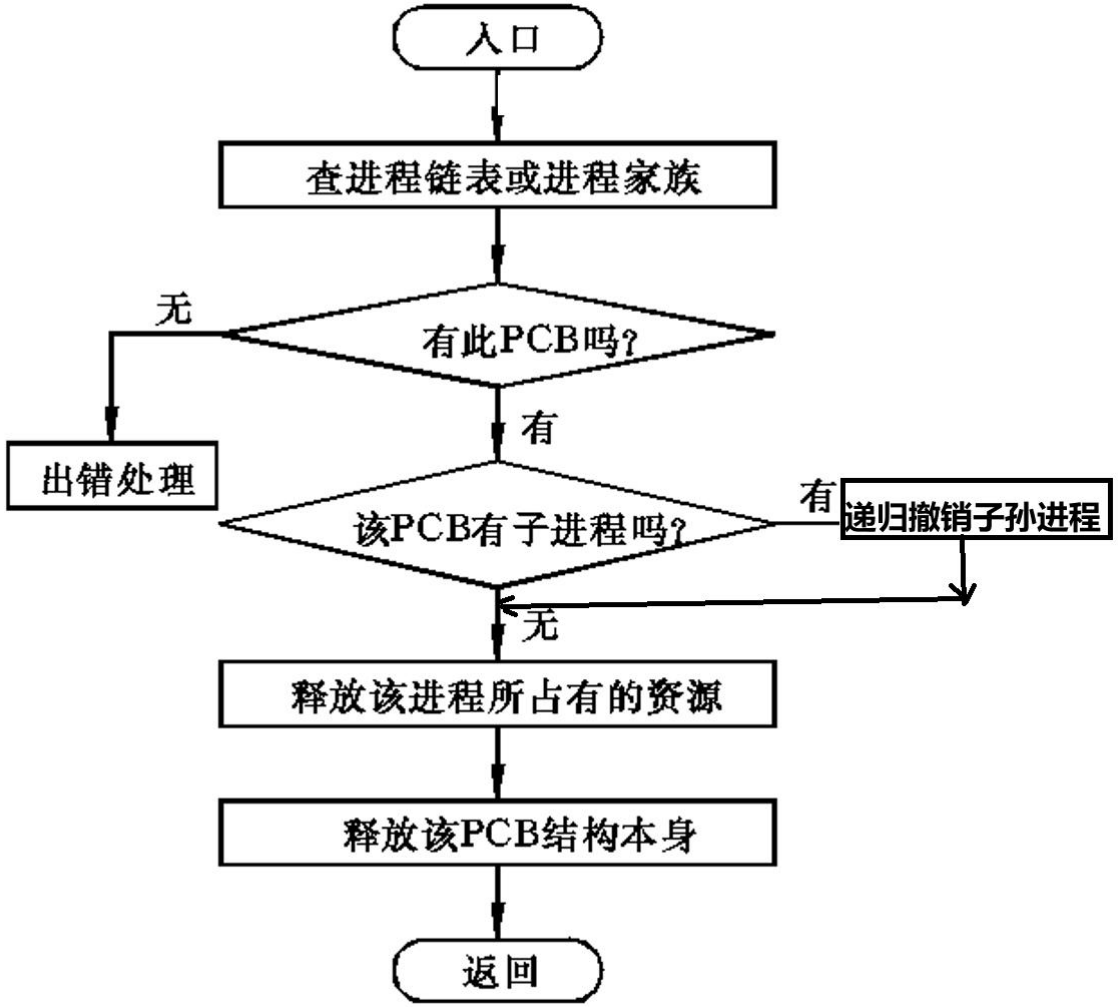
（3）初始化PCB。包括：

- 初始化标识信息
- 初始化处理机状态信息
- 初始化处理机控制信息：
- 程序计数器，
- 堆栈指针等进程状态——就绪或静止就绪、优先级等、

（4）将新进程插入就绪队列

3.3 进程控制

- 进程创建


入口
查进程链表或进程家族
有此PCB吗?
无
有
出错处理
该PCB有子进程吗?
有
递归撤销子孙进程
无
释放该进程所占有的资源
释放该PCB结构本身
返回

3.3 进程控制

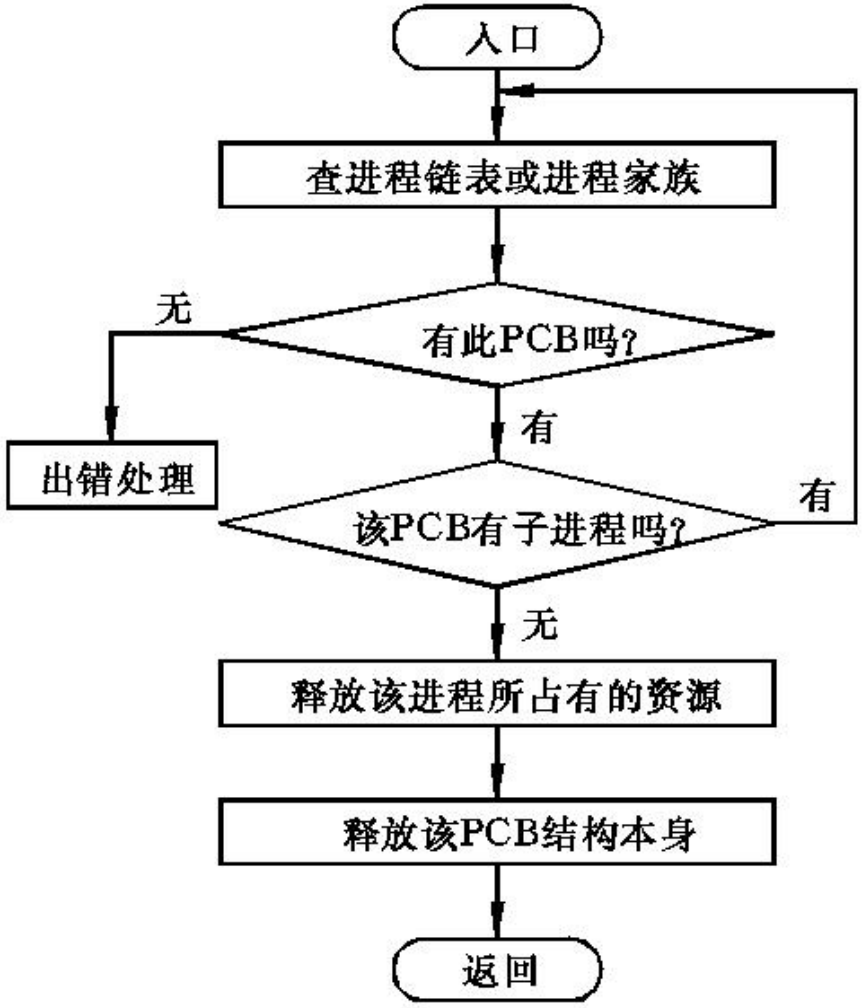
- 进程控制原语–进程终止Kill()
  - 引起进程终止的事件
    - 正常结束
    - 异常结束：越界错误、特权指令错、算术运算错等；
    - 外界干预：操作员或操作系统干预、父进程干预；

3.3 进程控制

- 进程控制原语-进程终止Kill()
  - 终止原语的处理过程
    - 找到PCB：根据终止进程标识符n，找PCB；
    - 检查进程状态：如果进程正处于执行状态，需要立即停止其执行；
    - 设置调度标志：撤销进程时，需要设置调度标志Sched
      - Sched=True；被撤消进程原处于运行状态；
      - Sched=False；被撤消进程原处于非执行状态；（思考为什么）
    - 撤消子孙进程：如果被撤销的进程有子孙进程，还需要递归地撤销这些子孙进程(如果没有收养机制的情况）。
    - 释放资源：回收被撤销进程占有的全部资源，包括内存、文件句柄、信号量等，并将这些资源归还给父进程或系统。
    - 释放PCB：最后，释放被撤销进程的PCB，并将其从系统中移除。

3.3 进程控制

- 进程终止(修改)



3.3 进程控制

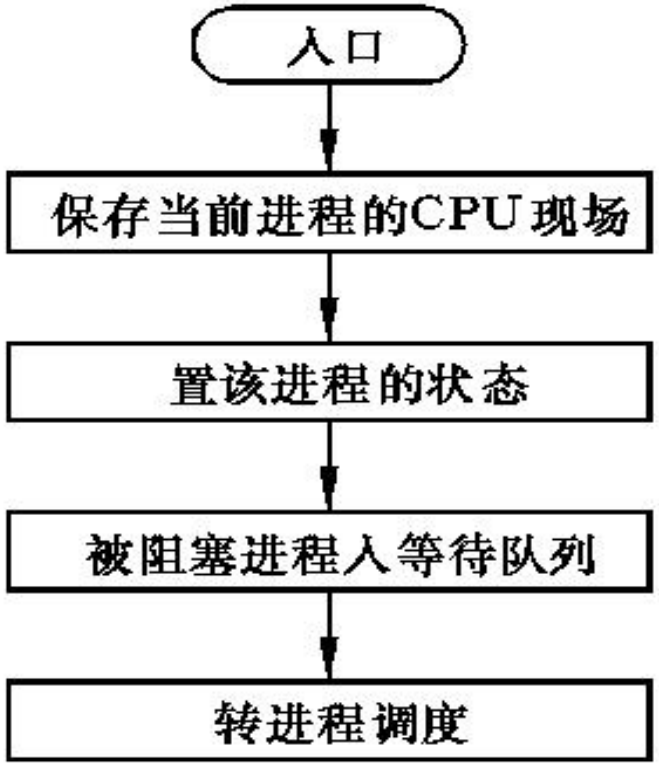
- 进程控制原语–进程阻塞block()
  - 引起进程阻塞的事件
    - 请求系统服务失败：
    - 启动某种操作并等待操作完成；
    - 等待合作进程间协同配合
    - 系统进程无新工作可做
    - 等待外部事件或新数据到达

3.3 进程控制

- 进程控制原语-进程阻塞Block()
  - 阻塞原语的处理过程
    - 找到PCB：根据阻塞进程标识符n，找PCB；
    - 保护现场：如果进程当前处于运行状态，系统需要保存该进程的CPU现场，即保存其程序计数器、寄存器等状态信息，以便在进程被唤醒后能够恢复执行。
    - 改变进程状态：将进程的状态由运行状态（Running）改为阻塞状态（Blocked）。这意味着进程不再占用CPU资源，等待某个条件的满足。
    - 插入阻塞队列：将进程的PCB插入到相应事件的等待队列中。不同的阻塞原因（如等待I/O完成、等待资源释放等）可能对应不同的等待队列。
    - 调度其他进程：调用进程调度程序，选择一个就绪进程进行执行。

3.3 进程控制

- 进程控制原语–进程阻塞Block()



3.3 进程控制

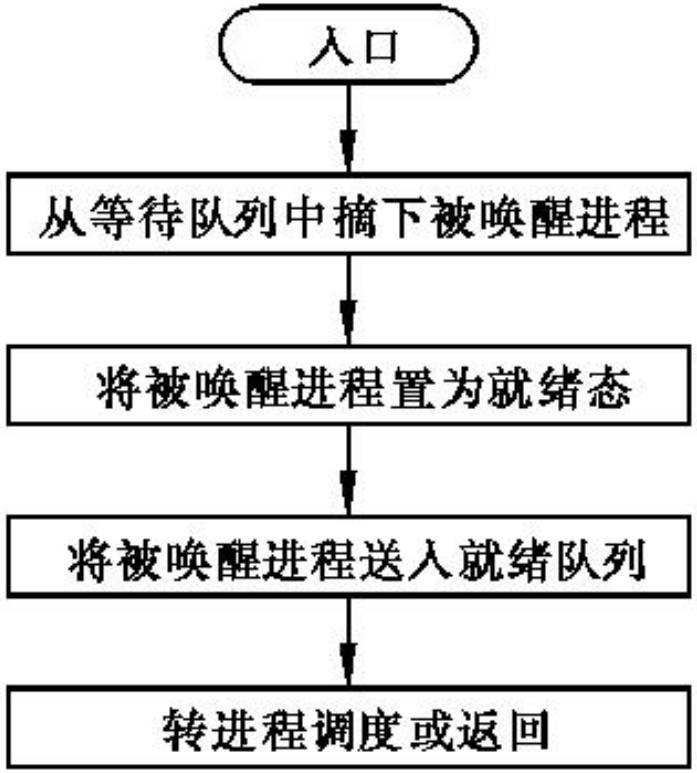
- 进程控制原语–进程唤醒wakeup()
  - 当等待队列中的进程所等待的事件发生时，会触发相应的唤醒机制，等待该事件的所有进程都将被唤醒。使进程从阻塞状态转变为就绪状态，等待CPU调度执行。
  - 引起进程唤醒的事件：
    - 主要包括I/O操作完成、资源获取、事件到达、新数据到达、系统服务满足、退出临界区以及有新工作可做等。
  - 唤醒一个进程可能由系统进程统一控制事件的发生并将“事件发生”这一消息通知等待进程，也可能由事件发生进程唤醒等待进程。

3.3 进程控制

- 进程控制原语-进程唤醒Wakeup()
  - 唤醒塞原语的处理过程
    - 识别阻塞进程：识别出因等待某个事件而处于阻塞状态的进程。这些进程通常会被放置在特定的阻塞队列中。
    - 移出阻塞队列：将被唤醒进程的PCB从阻塞进程队列移出，这是通过修改进程控制块（PCB）中的相关指针或链表结构来实现的。
    - 修改进程状态:将其PCB中进程状态由“阻塞”改为“就绪”
    - 插入就绪队列：将改PCB插入到就绪队列中
  - 唤醒原语完成唤醒任务后，唤醒原语的处理过程通常有两种选择：
    - 返回原调用程序：
    - 转向进程调度：

3.3 进程控制

- 进程控制原语–进程唤醒Wakeup()



3.3 进程控制

```
#include <stdio.h>
#include <unistd.h>
main()
{  int pid;
   pid=fork( );
  if (pid <0)
  { printf ("fork failed!\n");
    exit(1);}
   if( pid == 0)
   {printf("child process: pid is %d\n",pid);
     printf ("It is child process, pid is %d\n", getpid ());}
   else
   {sleep（）；
    printf("parent process:pid is %d\n",pid);
    printf("It is parent process pid is %d\n", getpid());
   }
    exit( 0 );
}
```

```
shenhl@shenhailan ~
$ gcc -o testfork testfork.c

shenhl@shenhailan ~
$ ./testfork
parent process:pid is 1836
child process: pid is 0
It is parent process pid is 2428
It is child process, pid is 1836

shenhl@shenhailan ~
```

3.3 进程控制

```
#include<sys/types.h>
#include<sys/wait.h>
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 int pid,tmp;
 int status;
 if( (pid = fork()) < 0 )
   printf("call fork() failed!\n");
 if (pid ==0)
   {printf("child process %d finished!\n",getpid( ));
    exit(1);}
  else
   {tmp=wait(&status);
    printf ("the tmppid   is %d\n",tmp);
    if ( tmp!= pid )
       printf("call wait() failed!\n");
     else
      printf( "parent process %d finished!\n",getpid());}
}
```

```
shenhl@shenhailan ~
$ ./testwait
child process 1828 finished!
the tmppid   is 1828
parent process 2744 finished!

shenhl@shenhailan ~
$
```

3.3 进程控制

实例：Linux系统进程控制函数

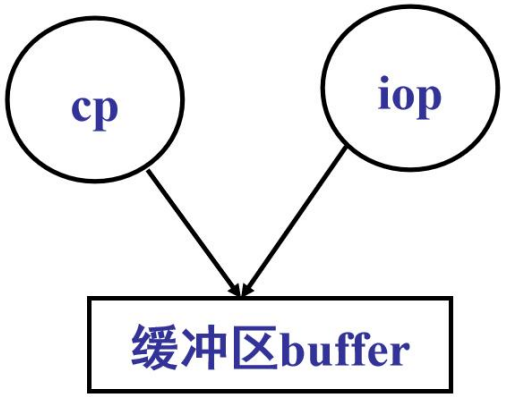
- pid_t fork(void);
- pid_t vfork(void);
- int clone(int (*fn)(void *), void *child_stack, int flags, void *arg, ...);
- void exit(int status);
- pid_t wait(int *status);
- pid_t waitpid(pid_t pid, int *status, int options);

3.3 进程控制

- 思考：
  - 如果系统中有N个进程，运行的进程最多几个，最少几个；就绪进程最多几个最少几个；等待进程最多几个，最少几个？
  - 有没有这样的状态转换，为什么？
    等待—运行；　　就绪—等待
  - 一个状态转换的发生，是否一定导致另一个转换发生？列出所有的可能
  - 试说明PCB的作用，为什么说PCB是进程存在的唯一标志？
  - 说明进程在三个基本状态之间转换的典型原因？

3.4 进程互斥与同步

- **并发进程之间的两种制约关系：**
  - 例1：A进程和B进程共享使用打印机。
  - 例2：生产者和消费者问题



  - 间接制约：PA进程和PB进程因共享某个临界资源而使各自的执行速度受到彼此的制约。并发进程间的间接制约关系。如：A进程和B进程共享使用打印机。
  - 直接制约：PA进程和PB进程在执行流程上存在着逻辑上的先后次序关系，进程的执行结果为对方的执行条件，从而限制各进程的执行速度。并发进程之间的的直接制约关系，如：生产者和消费者问题。

3.4 进程互斥与同步

- **并发进程之间的两种制约关系：**
  - 无论哪种制约关系，如果没有任何控制机制，PA和PB进程按照各自的处理流程会是：
    - PA() { ...;printer（）；...}
    - PB() { ...;printer（）;....}
  - 或者：
    - producer() { ...;sentdata（）；...}
    - Consumer() { ...;receivedata（）;....}
  - 并发执行环境下如果不加控制的话，就会产生对共享变量或数据结构等资源不正确的访问次序，导致错误的运行结果或每次运行结果的不一致。（与时间有关的错误）

3.4.1 并发进程间的制约关系

- **并发进程之间的两种制约关系：**
  - 在多道程序系统中，需要建立起一些机制，以保证多个进程“有条不紊地”进行：
  - 这些机制的主要任务是：
    - 对多个相关进程在执行次序上进行协调
    - 使并发执行的诸进程之间能按照一定的规则（或时序）共享系统资源，并能很好地相互合作，从而使程序地执行具有可再现性。

      互斥与同步控制问题

3.4.2 进程互斥

- 进程互斥的基本定义
  - 临界资源：在一段时间内只允许一个进程访问的资源，即仅当一个进程访问完并释放该资源后，才允许另一个进程访问的资源，称为临界资源或独占资源。临界资源可以是硬件，也可以是软件，变量、数据、表格等。
  - 临界区：把不允许多个并发进程交叉执行的一段程序称为临界部分（critical section ）或临界区（critical region）。临界区也可以被称为访问临界资源的那段程序。

3.4.2 进程互斥

- 进程互斥的基本定义
  - 互斥：一组并发进程中的一个或多个程序段，因共享某一临界资源而导致它们必须以一个不允许交叉执行的单位执行。
  - 不允许两个以上的共享该临界资源的并发进程同时进入临界区，这种间接制约关系称为进程的互斥。

3.4.2 进程互斥

- **进程互斥的基本定义**
  - 注意：临界区是对某一临界资源而言的，对于不同临界资源的临界区，它们之间不相交，所以不必互斥地执行；而相对于同一临界资源的若干个临界区，则必须互斥的进入，即对临界资源的操作必须互斥地执行。
  - 回答：例如有程序段A、B是关于变量X的临界区，而C、D是关于变量Y的临界区，那么，A、B之间（ 需要？不需要？ ）互斥执行，C、D之间（ 需要？不需要？ ）互斥执行，而A与C、B与D之间（ 需要？不需要？ ）互斥执行

3.4.2 进程互斥

- 怎样解决互斥问题

P1	P2
codesection11;	codesection21;
print;	print;
codesection12;	codesection22;

- 思考：互斥控制方案的设计思路，达到一些什么目标？

3.4.2 进程互斥

- 互斥执行的原则
  - 空闲让进：当无进程处于临界区时，表明临界资源处于空闲状态，应允许一个请求进入临界区的进程立即进入自己的临界区，以便有效地利用临界资源。
  - 忙则等待：当已有进程进入临界区时，表明临界资源正在被访问，因而其他试图进入临界区的进程必须等待，以保证对临界资源的互斥访问。
  - 让权等待：当进程不能进入自己的临界区时，应立即释放处理机，以免进程陷入“忙等”。——不忙碌等待。
  - 有限等待：对要求访问临界资源的进程，应保证在有限的时间内能进入自己的临界区，以免陷入“死锁”状态。——不死等。不互相阻塞。

3.4.2 进程互斥

- 软件方法解决互斥问题
  - 方法一　设置标志位flag[i]来标识Pi是否在临界段中，flag[i] 初值为false，表示Pi未使用临界资源。
  - Pi结构：

```
while (flag[j]) do skip;
    flag[i]=true;
        critical section;
    flag[i]:=false;
remainder section;
```

  - 问题：当flag[i], flag[j] 为false时，Pi, Pj 同时都想使用该临界资源，可能会出现什么样的结果？原因？
  - 启发：测试锁和关锁操作必须是连续的，不允许分开进行

3.4.2 进程互斥

- 软件方法解决互斥问题
  - 方法二 设一整型变量turn来标识何进程可进入临界区。初值设为：turn=i。
  - Pi结构：

```
while turn<>i do skip;
    critical section;
      turn:=j;
remainder section
```

  - 问题：若Pj未进入，而Pi还想进入时，会怎样？

3.4.2 进程互斥

- 软件方法解决互斥问题
  - 方法三（Peterson算法） 通过两个标志变量和一个共享变量来控制两个进程对临界区的访问，确保在任何时刻只有一个进程可以进入临界区。

Pi结构

```
bool flag[2] = {false, false};
int turn;
void Pi() {
    while (true) {
        flag[i] = true;
        turn = j;

        while (flag[j] && turn = = j);
          critical section;
         flag[i] = false;
         remainder section
    }
}
```

Pj结构

```
bool flag[2] = {false, false};
int turn;
void Pj() {
    while (true) {
        flag[j] = true;
        turn = i;

        while (flag[i] && turn = = i);
          critical section;
         flag[j] = false;
        remainder section
    }
}
```

3.4.2 进程互斥

- 硬件方法解决互斥问题-关中断
  - 在进入锁测试之前关闭中断，直到完成锁测试并上锁之后才能打开中断。
  - 进程在临界区执行期间，计算机系统不响应中断，也不会发生进程或线程调度，保证测试锁和关锁操作的连续性和完整性。
  - 缺点：
    - 滥用锁导致严重后果
    - 关中断时间过程，影响系统效率
    - 不适用于多CPU系统

3.4.2 进程互斥

- 硬件方法解决互斥问题-利用TS硬件指令
  - TS指令：Test-an-Set
  - 硬件指令的特点：指令操作仅需一个指令周期。
  - 描述：

```
Boolean TS(Boolean *lock)
{ Boolean old;
  old=*lock;
  *lock=TRUE;
  return old;
}
```

利用TS指令实现互斥

```
process Pi
do
{   .....
    while TS(lock);
    critical section;
    lock:=FALSE;
    remainder section;
}while (TRUE)
```

3.4.2 进程互斥

- 硬件方法解决互斥问题-利用SWAP硬件指令
  - SWAP指令：用于交换两个字的内容
  - 描述：void swap(boolean *a,boolean *b)
    ```
    { boolean temp;
      temp=*a; *a=*b; *b=temp;}
    ```
  - 利用SWAP指令实现互斥
    ```
    process Pi
    do
    {  key=TRUE
       do{
         swap(&lock,&key)// Lock赋值为TRUE，上锁
          }while(key!=FALSE)
           critical section;
           lock=FALSE;
           ...
       }while (TRUE)
    ```
    缺点：存在“忙等”

3.4.2 进程互斥

- 信号量方法解决互斥问题
  - 信号量机制是1965年，由荷兰计算机科学家Dijkstra提出的同步方法。
  - 信号量核心思想仍然是基于“检测锁”的思想，但是检测锁和锁状态的更新两者通过原语设计保证是连续执行不可分开的。
  - 信号量：仅能由互斥/同步控制原语对其进行操作的变量。
  - 互斥控制原语：P/wait操作；V/signal操作；
  - 信号量机制目前已经发展成多种类型，主要有：整型、记录型和信号量集机制。

3.4.2 进程互斥

- 信号量方法解决互斥问题-整型信号量
  - 信号量说明： var s: semaphore;
  - 原语描述

```
wait(s):   while s<=0 do skip;
                  s:=s-1;
signal(s):  s:=s+1;
```

  - 按用途，信号量可分为两大类：

    1、二元信号量：用作互斥变量，初值为1；

    2、一般信号量：用于一般同步，初值为共享资源初始数量；

3.4.2 进程互斥

- 信号量方法解决互斥问题-整型信号量
  - 利用整型信号量实现进程互斥
  - Var mutex:semaphore:=1;

```
P1: repeat                          P2: repeat
      wait(mutex);                        wait(mutex);
      critical section;                   critical section;
      signal(mutex);                      signal(mutex);
      remainder section;                  remainder sect;
    until false;                        until false;

parbegin
  P1;
  P2;
parend
```

3.4.2 进程互斥

- 信号量方法解决互斥问题-记录型信号量（计数型）
  - 引入目的：克服整型信号量机制存在的“忙等” (Busy waiting)现象，有效地提高资源的利用率。
  - 结构描述：type semaphore=record
    value:integer;
    L:list of process;
    end

```
wait(s);
{semaphore s;
   s.value =s.value-1;
   if   s.value<0
   then {  insert(caller,s.L);
           block(caller);    }
}
```

```
signal(s);
{semaphore s;
   s.value =s.value+1;
   if   s.value<=0
   then {  remove(s.L,id);
           wakeup(id);    }
}
```

3.4.2 进程互斥

- 信号量方法解决互斥问题-记录型信号量（计数型）
  - 信号量的物理意义
  - Wait(s)操作：

1）请求分配一个S代表的资源，执行s.value-1；

2）若s.value<0，表示系统已无该类资源，申请者阻塞。

  - Signal(s)操作：

1）进程释放一个S代表的资源，执行s.value+1；

2）若s.value<=0，表示尚有进程因等待S代表的资源而处于阻塞状态，所以应唤醒其中之一。

3.4.2 进程互斥

- 信号量方法解决互斥问题-记录型信号量（计数型）
  - 利用信号量实现进程互斥

```
Var mutex:semaphore:=1;
parbegin
        P1: repeat                      P2: repeat
            wait(mutex);                    wait(mutex);
            critical section;               critical section;
            signal(mutex);                  signal(mutex);
            remainder section;              remainder sect;
            until false;                    until false;

                                            parend
```

3.4.2 进程互斥

■ 信号量方法解决互斥问题-记录型信号量（计数型）

■ 利用信号量实现进程互斥

```
Var mutex:semaphore:=1;
parbegin
P1:
do
   {①wait(mutex);
    ②critical section;
    ③signal(mutex);
    remainder section; }
while (TRUE)
P2:
do
   {①wait(mutex);
    ②critical section;
    ③signal(mutex);
    remainder section;
}while(TRUE)
P3:
do
   {①wait(mutex);
    ②critical section;
    ③signal(mutex);
    remainder section;
}while(TRUE)
parend
```

时间片	执行语句	mutex.value	mutex.L
	初始	1	NULL
t1	P1①	0	NULL
t2	P1②	0	NULL
t3	P2①	-1	P2
t4	P3①	-2	P2, P3
t5	P1③	-1	P3
t6	P2②	-1	P3
t7	P2③	0	NULL
t8	P3②	0	NULL
t9	P3③	1	NULL

思考如下问题：

（1）如果同时允许有两个进程使用该资源，mutex.value的初始值设为多少？

（2）假设是互斥资源，如果mutex.value=1,则当前有进程在使用该资源吗？

mutex.value=0表示什么？mutex.value为负值时，|mutex.value|表示什么？

3.4.2 进程互斥

- 练习：
  - 现有n个具有相关临界区的并发进程，如果某进程调用P操作后变为等待状态，则调用P操作时信号量的值必定为（ ）。

    A．≤0　　B．1　　C．n-1　　D．n
  - 用PV操作管理临界区时把信号量的初值定义为1，现已有一个进程在临界区，但有n个进程在等待进人临界区，这时信号量的值为（ ）。

    A．-1　　B．1　　C．-n　　D．n
  - 有m个进程共享同一临界资源，若使用信号量机制实现对资源的互斥访问，则信号量值的变化范围是_____
  - 设与某资源关联的信号量初值为3，当前值为1。若M表示该资源的可用个数，N表示等待该资源的进程数，则M、N分别是___

3.4.2 进程互斥

- 信号量方法解决互斥问题-AND型信号量集
  - 问题提出：两个进程A和B，它们都要求访问共享临界数据D和E。为此，为这两个数据分别设置用于互斥的信号量Dmutex和Emutex，并令它们的初值都是1。相应地，在两个进程进入临界区前都要包含两个对Dmutex和Emutex的操作，即：

(1)

```
process A:          process B:
    wait (Dmutex);      wait(Dmutex);
    wait (Emutex);      wait(Emutex);
```

或

(2)

```
process A:          process B:
    wait (Dmutex);      wait(Emutex);
    wait (Emutex);      wait(Dmutex);
```

  - 第一种情况：满足互斥执行
  - 第二种情况：若进程A和B按下述次序交替执行wait操作，会出现什么情况？
    - process A: wait(Dmutex);
    - process B: wait(Emutex);
    - process A: wait(Emutex);
    - process B: wait(Dmutex);

3.4.2 进程互斥

- 信号量方法解决互斥问题-AND型信号量集
  - 称为同时wait操作，即swait(simultaneous wait)。目的：避免死锁

```
swait(Semaphore s1,s2,...,sn)
 {if(s1.value>=1 && s2.value>=1&&...&&sn.value>=1)
    for(int i=1;i<=n;i++) si.value=si.value-1;
  else
    for(int i=1: si.value>=1;i++)//寻找第一个不满足的si队列;
     将进程插入第一个不满足的si.L队列;
     进程的程序计数重置为swait操作起点;
     阻塞进程;
  }
```

3.4.2 进程互斥

- 信号量方法解决互斥问题-AND型信号量集
  - 称为同时wait操作，即swait(simultaneous wait)。
  - 目的：避免死锁

```
ssignal(semaphore s1,s2,...,sn)
 { for(int i=1; i<=n;i++)
     si.value=si.value+1;
     唤醒si.L中所有阻塞进程;
 }
```

3.4.2 进程互斥

- 信号量方法解决互斥问题--一般信号量集
  - 一次可给进程分配多种临界资源，且同类临界资源一次可分配多个。

```
swait(semaphore s1,int t1,int d1,...,semaphore sn,int tn,int dn)
 {if(s1.value>=t1 && s2.value>=t2&&...&&sn.value>=tn)
     for(int i=1;i<=n;i++) si.value=si.value-di;
       else
        for(int i=1: si.value>=ti;i++)//寻找第一个不满足的si队列;
        将进程插入第一个不满足的si.L队列;
        进程的程序计数重置为swait操作起点;
        阻塞进程;
      }
```

3.4.2 进程互斥

- 信号量方法解决互斥问题--一般信号量集
  - 一次可给进程分配多种临界资源，且同类临界资源一次可分配多个。

```
ssignal(semaphore s1,int d1;...,semaphore sn,int dn)
    { for(int i=1; i<=n;i++)
      si.value=si.value+di;
      唤醒si.L中所有阻塞进程;
    }
```

3.4.2 进程互斥

- 信号量方法解决互斥问题--一般信号量集
- 一般信号量集的几种特殊情况

  swait(s,d,d)：每次申请d个资源，资源数小于d时不予分配；

  swait(s,1,1)：蜕化为wait(s)；

  swait(s,1,0)：成为可控开关：s>=1时，所有进程都可进入；

  s<=0时，阻止所有进程进入；

3.4.3 进程同步

- **进程同步的基本定义**
  - **进程同步：**异步环境下的一组并发进程，因**直接制约**而互相发送消息而进行**互相合作、互相等待**，需要各进程按一定的速度执行的过程称为**进程间的同步**；具有同步关系的一组并发进程称为**合作进程**
  - **互斥与同步的区别**：进程互斥时各进程的执行顺序是任意的，谁先获得资源谁先进入临界区。进程同步时各进程执行顺序本身存在着逻辑先后关系。

3.4.3 进程同步

- 信号量解决同步问题
  - 思考下面一些问题：
    - 不加任何同步控制会怎样？
    - 同步控制的目标？
    - 怎么实现这一目标？

3.4.3 进程同步

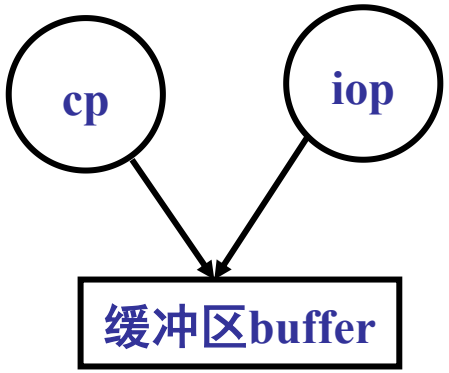
- 信号量解决同步问题
  - 思考下面一些问题：
    - 不加任何同步控制会怎样？
    - 同步控制的目标？正确发送和接收，不空等也不盲目推进
    - 怎么实现这一目标？取/放数据之前测试是否满足取/放的前提条件，具备则开始往下执行，否则阻塞进入等待状态
    - 这个前提由谁发出？对方进程
    - 前提条件的表达？消息或者转化为某种资源情况的表达
    - 一种最为简单和直观的方法是直接制约的进程互相给对方进程发送执行条件已经具备的信息。

3.4.3 进程同步

- 信号量解决同步问题
  - 把各进程之间发送的消息（或对应的某种资源）作为信号量看待。
  - 与进程互斥时不同的是，这里的信号量只与制约进程及被制约进程有关而不是与整组并发进程有关。因此，称该信号量为私用信号量。
  - 一个进程Pi的私用信号量Semi是从制约进程发送来的进程Pi的执行条件所需要的消息或资源。
  - 与私用信号量相对应，称互斥时使用的信号量为公用信号量。

3.4.3 进程同步

- 信号量解决同步问题
  - 例题1：设某计算进程cp和打印进程iop共用一个单缓冲区
  - cp进程计算数据并送入缓冲区
  - iop进程从缓冲区取数据并打印



  - 其中要求：
    - 当cp进程把计算结果送入缓冲区buffer后，iop进程才能从缓冲区buffer中取出结果去打印 。
    - 当iop进程把缓冲区buffer中的数据取出打印后，cp进程才能把下一个计算结果送入缓冲区buffer中。

3.4.3 进程同步

- 信号量解决同步问题

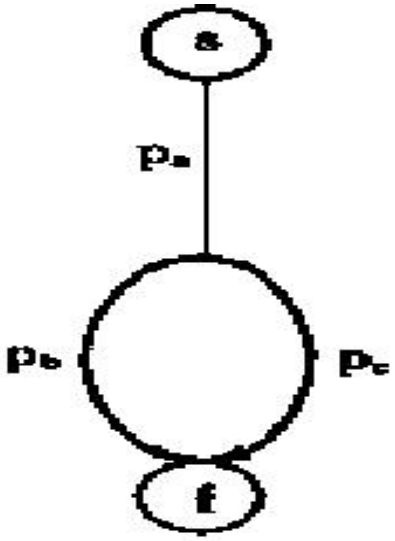
```
main ( )
 { semaphore
     buffull=0;
   semaphore
     bufempty=1;
   cobegin
     cp( );
     iop( );
   coend;
}
```

```
void cp( ) {
   while (计算未完成)
   {
   得到一个计算结果；
   wait(bufempty);
   将计算结果送缓冲区
     buffer中；
   signal(buffull);
}
}
```

```
void iop( ) {
   while (打印工作未完成)
   {
   wait (buffull);
   从缓冲区buffer中取出
     信息；
   signal ( bufempty);
   打印输出结果；
}
}
```

3.4.3 进程同步

- 信号量解决同步问题
  - 例题2：进程pa, pb, pc为一组合作进程，其进程流图如右图所示，使用信号量机制来实现这三个进程的同步。




```
main( )
{ semaphore
    SB=SC=0;
  cobegin
    pa();
    pb();
    pc();
coend;
}
```

```
void  pa()
{
  ……
  signal(SB);
  signal(SC);
}
```

```
void  pb()
{
  wait(SB);
  ……
}
```

```
void  pc()
{
  wait(SC);
  ……
}
```

3.4.4 经典进程的互斥和同步问题

- 生产者–消费者问题
- 读者–写者问题
- 哲学家进餐问题
- 理发师问题

3.4.4 经典进程的互斥和同步问题

- 生产者–消费者问题



- 多个生产者进程：往缓冲区放数据
- 多个消费者进程：从缓冲区取数据
- 缓冲区数目：n
- 各生产者进程和消费者进程对缓冲区的访问需互斥

3.4.4 经典进程的互斥和同步问题

- 生产者-消费者问题
  - 分析各进程之间的关系
    - 生产者-生产者？生产者-消费者？消费者-消费者？
      消费者-生产者
  - 初始化：
    - 整型变量in，out:放数据和取数据时的缓冲区单元指示
    - 信号量empty：记录空缓冲区数目，用于生产者执行放数据的前提，初值为n
    - 信号量full：记录满缓冲区数目，用于消费者执行取数据的前提，初值为0
    - 信号量mutex:用于生产者和消费者对缓冲区数据操作的互斥

生产者–消费者问题

```
void producer_i( ) (i=,2,…,k)
//生产者进程
{
    item nextp ;
    do
   {  …
     produce an dataP ;  //生产数据dataP
       …
     wait(empty) ;  //测试
     wait(mutex);  //互斥
     buffer[in] = dataP ;
     in = (in + 1)% n ;
     signal(mutex) ;//成对
     signal(full) ;   //交叉成对
  }while (TRUE);
}
```

```
void consumer_j ( ) (j=1,2,…,m）
{  item nextc ;
   do {
     wait(full);
     wait(mutex);
     dataC = buffer[out];  //放入数据dataC
     out = (out + 1)% n ;
     signal(mutex);
     signal(empty);
     consume the item in dataC ;
   }while (TRUE) ;
}
```

```
void main()
{ Cobegin
  producer_i( )(i=1,2,3,...,k);    consumer_j( )  (j=1,2,…,m）
  Coend}
```

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题

  - 问题说明：五个哲学家就餐
  - 哲学家就餐事务流程：（1）eating；　（2）thinking；
  - 约束限制：每个哲学家在eating之前需要拿到左右两个筷子参能进行eating活动，eating完成后放下两根筷子

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 分析各进程之间的关系
    - 哲学家进程-哲学家进程？
  - 初始化：
    - semaphore chopstick[5]；代表5根筷子的资源，用作相邻哲学家进程的互斥，初值均为1

哲学家就餐问题

```
void  Pi( )            (i = 0，1，2，3)    //前4个哲学家
{
   while (true)
   {  wait (chopstick[i]);        //拿起左边筷子
      wait (chopstick[i + 1]);    //拿起右边筷子
      eating ;                          //吃面条
      signal (chopstick[i]);      //放下左边筷子
      signal (chopstick[i+1]);    //放下右边筷子
      thinking ;                        //思考
   }
}
```

```
void  P4 ( )          //第5个哲学家
                                  {
   while (true)
   {  wait (chopstick[4]);       //拿起左边筷子
      wait (chopstick[0]);       //拿起右边筷子
      eating ;                   //吃面条
      signal (chopstick[4]);     //放下左边筷子
      signal (chopstick[0]);     //放下右边筷子
      thinking ;                 //思考
   }
}
parend
```

```
void main()
{ parbegin
   Pi( )    (i=1,2,3,...,5);
  parend
```

思考：此算法虽然能保证相邻哲学家对筷子的访问互斥，但可能引起？

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 对上述哲学家就餐问题算法的死锁问题，可采取下面几种解决方法之一：
    - 至多允许4个哲学家同时取左边的筷子，这样能至少保证一个哲学家能就餐。
    - 仅当哲学家左右两只筷子均可用时，才允许他拿起筷子进餐。
    - 规定奇数号哲学家先拿左边筷子，然后再拿右边筷子；而偶数号哲学家先拿右边筷子，然后再拿左边筷子。
    - 规定每个哲学家先拿序号小的筷子——按序号分配。
    - 同一时间最多允许1个哲学家进餐(即进餐互斥)。此算法并发程度最差。

3.4.4 经典进程的互斥和同步问题

- **哲学家就餐问题：**

方法一：规定至多允许4个哲学家同时取左边的筷子。

```
semaphore chopstick[5], S;
chopstick[0].value=chopstick[1].value=1;
chopstick[2].value=chopstick[3].value=1 ;chopstick[4].value=1 ;
S.value=4;      //最多允许4个哲学家同时拿筷子
// 以下是5个哲学家进程代码的统一写法
void  Pi( ) (i = 0, 1, 2, 3, 4)  //第i个哲学家进程
{  while (true)
   {   wait (S);                          // 看看是否允许拿筷子
       wait (chopstick[i]);               //拿起左边筷子
       wait (chopstick[(i+1)%5]);         //拿起右边筷子
       eating ;                           //吃面条
       signal (chopstick[i]);             //放下左边筷子
       signal (chopstick[(i+1)%5]);       //放下右边筷子
       signal (S);
       thinking ;                         //思考
   }
}
void main()
{ parbegin
   Pi( )      (i=1,2,3,...,5);
  parend
```

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 方法二：用AND信号量机制。

```
// 初始化
semaphore chopstick[5] ;
chopstick[0].value=chopstick[1].value=1;
chopstick[2].value=chopstick[3].value=1 ;
chopstick[4].value=1 ;
//以下是5个哲学家进程代码的缩写，代表5段代码
void  Pi( )        (i=0, 1, 2, 3, 4)
{
   while (true)
   {
      int j=(i+1)%5;
      Swait (chopstick[i],chopstick[j])；  //拿起左右筷子
      eating ;      //吃面条
      Ssignal(chopstick[i],chopstick[j]); //放下左右筷子
      thinking ;   //思考
   }
}
void main()
{parbegin  P0 ();   P1 ();   P2 ();   P3 ();   P4 ();  parend}
```

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 方法三：奇数号哲学家先拿左边筷子，然后再拿右边筷子；而偶数号哲学家先拿右边筷子，然后再拿左边筷子。

```
semaphore chopstick[5] ;
chopstick[0].value=chopstick[1].value=1;
chopstick[2].value=chopstick[3].value=1 ;
chopstick[4].value=1 ;
parbegin
// 以下是3段代码的缩写，代表3个进程
void  Pi( ) (i = 0, 2, 4)   //偶数号哲学家的活动
{
   while (true)
   {
      int j=(i+1)%5;
      wait (chopstick[i+1]);      //拿起右边筷子
      wait (chopstick[i]);       //拿起左边筷子
      eating ;                   //吃面条
      signal (chopstick[i+1]);   //放下右边筷子
      signal (chopstick[i]);     //放下左边筷子
      thinking ;                 //思考
   }
```

```
void  Pi( )  (i=1, 3)      //奇数号哲学家的活动
{
   while (true)
   {  wait (chopstick[i]);          //拿起左边筷子
      wait (chopstick[i+1]);       //拿起右边筷子
      eating ;                     //吃面条
      signal (chopstick[i]);       //放下左边筷子
      signal (chopstick[i+1]);     //放下右边筷子
      thinking ;                   //思考
   }
}
void main()
{parbegin
P0 ();  P1 ();  P2 ();  P3 ();  P4 ();
parend
}
```

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 方法四：规定每个哲学家先拿序号小的筷子(按序号分配)

```
semaphore chopstick[5] ;
chopstick[0].value=chopstick[1].value=1;
chopstick[2].value=chopstick[3].value=1 ;
chopstick[4].value=1 ;
// 以下是4段代码的缩写，代表4个进程
void  Pi( )          (i = 0, 1, 2, 3)          //前4个哲学家
{
   while (true)
   {
      wait (chopstick[i]);          //拿起左边筷子
      wait (chopstick[i+1]);        //拿起右边筷子
      eating ;                      //吃面条
      signal (chopstick[i]);        //放下左边筷子
      signal (chopstick[i+1]);      //放下右边筷子
      thinking ;                    //思考
   }
}
```

```
void  P4( )                                  //第5个哲学家
{
   while (true)
   {
      wait (chopstick[0]);          //拿起右边筷子
      wait (chopstick[4]);          //拿起左边筷子
      eating ;                      //吃面条
      signal (chopstick[0]);        //放下右边筷子
      signal (chopstick[4]);        //放下左边筷子
      thinking ;                    //思考
   }
}
void main()
{
parbegin
P0 ();  P1 ();  P2 ();  P3 ();  P4 ();
parend
}
```

3.4.4 经典进程的互斥和同步问题

- 哲学家就餐问题
  - 方法五：同一时间最多允许1个哲学家进餐(即进餐互斥)

```
// 初始化
同前
// 以下是5段代码的缩写，代表5个进程代码
void  Pi ()  (i = 0, 1, 2, 3, 4)            //第i个哲学家的活动
{
   while (true)
   {
      int j=(i+1)%5;
      wait (S);   // 互斥(不允许1个以上哲学家同时进餐)
      wait (chopstick[i]);      //拿起左边筷子
      wait (chopstick[j]);      //拿起右边筷子
      eating ;                  //吃面条
      signal (chopstick[i]);    //放下左边筷子
      signal (chopstick[j]);    //放下右边筷子
   signal (S);         // 允许其他哲学家进餐
      thinking ;                //思考
   }
}
```

```
void main ()
{parbegin  P0 ();  P1 ();  P2 ();  P3 ();  P4 ();  parend}
```

3.4.4 经典进程的互斥和同步问题

- 读者-写者问题
  - 问题说明：
    - 一个数据文件或记录，可被多个进程共享，我们把只要求读该文件的进程称为“读者进程”，其他进程称为“写者进程”。
  - 约束
    - 允许多个读者进程同时读一个共享文件
    - 不允许两个或两个以上写者进程同时访问共享文件
    - 不允许一个写者进程和其他读者进程同时访问共享文件

3.4.4 经典进程的互斥和同步问题

- 读者-写者问题
  - 分析各进程之间的关系
    - 写者-写者？写者和读者？读者-读者？
  - 初始化：
    - 信号量Wmutex：用来写者-写者以及写者-读者互斥，初值为1；

      思考：是否满足约束了？（看下图）

3.4.4 经典进程的互斥和同步问题

- 读者–写者问题
  - 下面满足约束吗？

```
semaphore Wmutex;
Wmutex.value=1;
void Reader_i  () (i = 1, 2, …) //读者进程
   {wait(Wmutex);
    Reading;
    signal(Wmutex);
    }
 void Writer_j  ()  (j = 1, 2, …) //读者进程
   {wait(Wmutex);
    Writing;
     signal(Wmutex);
    }
void main ()
{parbegin
 Reader_i  ()  (i = 1, 2, …);
Writer_j  ()  (j = 1, 2, …) ;
parend}
```

3.4.4 经典进程的互斥和同步问题

- 读者–写者问题
  - 完整算法

```
semaphore Wmutex，Rmutex;
int  Rcounter = 0;  //读者计数变量
Wmutex.value=1;  //写-写互斥、读-写互斥
Rmutex.value=1;   //用于Rcounter互斥
void Reader_i   ()  (i = 1, 2, …)
{
   wait(Rmutex);
   if(Rcounter==0) wait(Wmutex);
   Rcounter = Rcounter + 1;
   signal(Rmutex);
   Reading;
   wait(Rmutex);
   Rcounter = Rcounter – 1;
   if(Rcounter==0) signal(Wmutex);
   signal(Rmutex);
} //读者进程结束
```

```
void Writer_j  () (j=1, 2, …) {
   wait(Wmutex);
   Writing;
   signal(Wmutex);
}

void main ()
{parbegin
 Reader_i  ()  (i = 1, 2, …);
 Writer_j  ()  (j = 1, 2, …) ;
parend}
```

3.4.4 经典进程的互斥和同步问题

读者-写者问题

- 拓展思考：
  - 上述算法实际上是“优先读者”算法，当有读者正在读，且后续读者源源不断到来时，写者将长期得不到服务。写者“饿死”。
  - 为此，可以考虑“优先写者”的算法——当有写者要写时，待目前正在读的读者读完后，立即让写者去写（即一旦有写者到达，后续的读者都必须等待，而无论是否有读者在读文件）

3.4.4 经典进程的互斥和同步问题

- 读者-写者问题
  - “写者优先”算法：增加一个信号量w，用于在写进程到达时封锁后续的读者进程。

```
semaphore Wmutex，Rmutex;
int  Rcounter = 0;  //读者计数变量
Wmutex.value=1;  //写-写互斥、读-写互斥
Rmutex.value=1;   //用于Rcounter互斥
w.value=1 //用于写者优先
void Reader_i ()   (i = 1, 2, …) //读者进程
{   wait(w) ;
    wait(Rmutex); //准备访问共享变量Rcounter
    if(Rcounter==0) wait(Wmutex);
    Rcounter = Rcounter + 1;
    signal(Rmutex);
    signal(w);
     Reading;
    wait(Rmutex);
    Rcounter = Rcounter – 1;
    if(Rcounter==0) signal(Wmutex);
    signal(Rmutex);
} //读者进程结束
```

```
void Writer_j ()(j=1, 2, …)  //写者进程
{
    wait(w)
    wait(Wmutex);  //写-写互斥、写-读互斥
    Writing;
    signal(Wmutex);
    signal(w);
}

void main ()
{parbegin
 Reader_i  ()  (i = 1, 2, …);
 Writer_j  ()  (j = 1, 2, …) ;
parend}
```

3.4.4 经典进程的互斥和同步问题

- 读者-写者问题
  - “写者优先”算法：增加一个信号量W，用于在写进程到达时封锁后续的读者进程。

3.4.4 经典进程的互斥和同步问题

```
semaphore Wmutex，Rmutex;
int  Rcounter = 0;  //读者计数变量
Wmutex.value=1;  //写-写互斥、读-写互斥
Rmutex.value=1;  //用于Rcounter互斥
w.value=1 //用于写者优先
void Reader_i ()   (i = 1, 2, …) //读者进程
{
   wait(Rmutex); //准备访问共享变量Rcounter
   if(Rcounter==0) wait(Wmutex);
   Rcounter = Rcounter + 1;
   signal(Rmutex);

    Reading;
   wait(Rmutex);
   Rcounter = Rcounter – 1;
   if(Rcounter==0) signal(Wmutex);
   signal(Rmutex);
} //读者进程结束
```

```
void Writer_j ()(j=1, 2, …)  //写者进程
{
   wait(Rmutex)
   wait(Wmutex);  //写-写互斥、写-读互斥
   Writing;
   signal(Wmutex);
   signal(Rmutex);
}

void main ()
{parbegin
 Reader_i  ()  (i = 1, 2, …);
 Writer_j  ()  (j = 1, 2, …) ;
parend}
```

3.4.4 经典进程的互斥和同步问题

- 理发师问题
  - 问题描述：
    - 理发店有一位理发师，包含一间接待室和一间工作室，接待室内有n(n≥1)把椅子，而工作室只有1把椅子。
    - 如果没有顾客，理发师就去睡觉。
    - 如果理发师在忙且接待室有空闲椅子，那么此顾客会坐在其中1把空闲的椅子上等待；如果来时所有椅子都有人，那么顾客离去。
    - 如果理发师在睡觉；则顾客会唤醒他。请采用信号量机制解决该理发师问题。

3.4.4 经典进程的互斥和同步问题

- 理发师问题
  - 分析各进程之间的关系：
    - 理发师-顾客；顾客-顾客
  - 初始化
    - 控制变量count：用来记录等待理发的顾客数，初值为0；当count=n时，新来的顾客离去。
    - 信号量customers：用来记录等候理发的顾客数，并用来阻塞理发师进程，初值为0；
    - 信号量barbers：用来记录等候顾客的理发师数，并用来阻塞顾客进程，初值为1；
    - 信号量mutex：用于对共享变量count的互斥访问，初值为1

3.4.4 经典进程的互斥和同步问题

- 理发师问题

```
void customers  ():
 arrival();
 P(mutex);
  if (count<n)
{
    count := count+1;
    V(mutex);
    V(customers);
    P(barbers);
    get_cuthair();
  }
  else {
        V(mutex);
        exit();}
}
```

```
int count = 0 ;
semaphore barbers,customers,mutex ;
barbers.value = 0;
customers.value=0;
mutex.value=1;

void barbers ()
{
 while (true)  {
        P(customers);
        P(mutex);
        count := count - 1;
        V(mutex);
        cut_hair()
        V(barbers);
  }
```

```
void main ()
{parbegin
 customers  ()  (i = 1, 2, …,  n);
  barbers () ;
parend}
```

3.4.4 经典进程的互斥和同步问题

- 理发师问题
  - 拓展思考：
    - 有多个理发师的问题
    - 考虑公平性问题——顾客先来先服务
    - 除了接待室（有椅子）还有接待区（站着等候，人数有限制）的问题等等

3.4.6 实例

实例：Linux系统进程互斥/同步控制函数

POSIX信号量

- int sem_init(sem_t *sem, int pshared, unsigned int value);
- int sem_destroy(sem_t *sem);
- int sem_wait(sem_t *sem);
- int sem_trywait(sem_t *sem)
- int sem_post(sem_t *sem);
- int sem_getvalue(sem_t *sem, int *sval);

SYSTEM V信号量

- int semop(int semid, struct sembuf *sops, size_t nsops);

3.4.5 管程

- 管程基本概念
  - 背景
    - 信号量机制方便、有效，但每个要访问临界资源的进程都必须自备同步操作wait(S)和signal(S)，使大量的同步操作分散在各个进程中
      - 给系统管理带来麻烦
      - 会因同步操作使用不当而导致系统死锁
    - 汉森(Brinch Hanson)和霍尔(Hoare)提出了一种新的进程同步机制——管程(Monitors)

3.4.5 管程

- 管程的定义（Hason）
  - 管程定义了一个数据结构和能被并发进程执行(在该数据结构上)的一组操作，这组操作能同步进程和改变管程中的数据。通过管程，可以确保在并发环境下，一次只有一个进程或线程能够访问共享资源，从而避免数据竞争和不一致性。
- 管程的组成
  - 局部于管程的共享数据和条件变量说明
  - 对该数据结构进行操作的一组过程
  - 对局部于管程的数据设置初值的语句
  - 此外，还须为管程赋予一个名字

3.4.5 管程

- 管程说明
  - 局部于管程的数据结构，仅能被局部于管程的过程访问，任何管程外的过程都不能访问它。
  - 局部于管程的过程仅能访问管程内的数据结构。
  - 管程把共享变量和对它的操作的若干过程“封装”起来，所有进程要访问临界资源时，都必须经过管程才能进入，而管程每次只准许一个进程进入管程，从而实现了进程互斥。

3.4.5 管程

- 管程语法说明

```
type <管程名>=monitor
{
<共享变量说明>;
<条件变量说明>;
procedure <过程名1>(<形参表>)
{ <过程体1> }
      ……
procedure <过程名n>(<形参表>)
{ <过程体n> }
<过程局部数据初始化语句>;
}
```

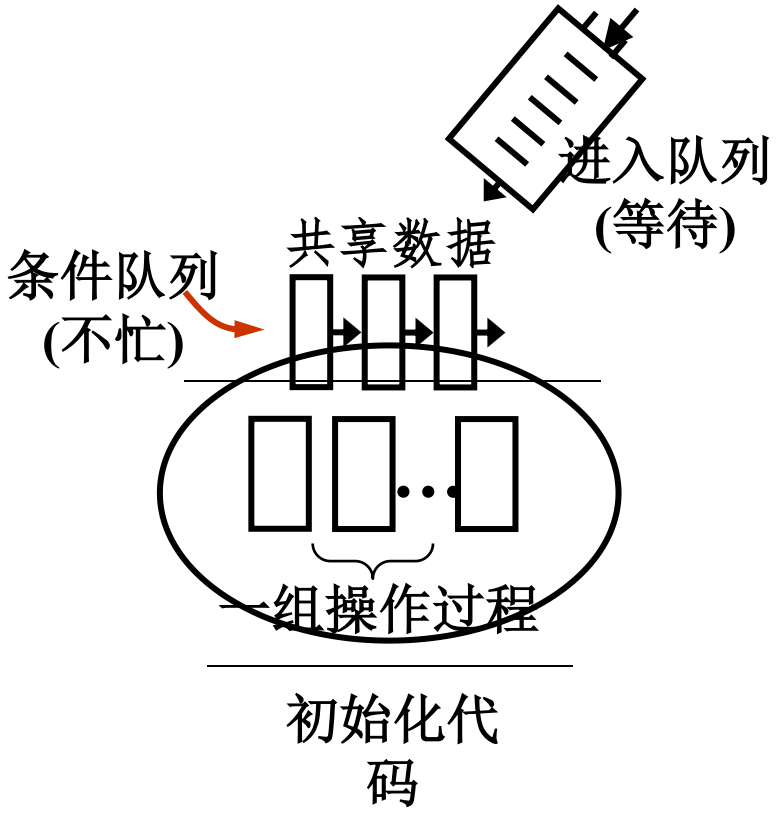


图3-8 管程示意图

3.4.5 管程

- 管程的条件变量
  - 利用管程实现进程同步时，必须设置两个同步操作原语wait和signal：
  - 某进程通过管程请求临界资源未能满足时，管程调用wait原语将该进程阻塞，并把它排到等待队列上。
  - 当另一进程访问完成并释放该资源后，管程调用signal原语唤醒等待队列中的队首进程。
  - 等待的原因可有多个，为区别它们，引入了条件变量condition。条件变量的说明形式为：

```
var x,y:condition
```

3.4.5 管程

- 管程的条件变量
  - 对应的原语调用形式为：x.wait和x.signal
    - x.wait原语:正在调用管程的进程因x条件需要被阻塞或挂起，则调用x.wait将自己插入到x条件的等待队列上，并释放管程，直到x条件变化，此时其他进程可以使用该管程。
    - x.signal:正在调用管程的进程发现x条件发生了变化，则调用x.signal,重新启动一个因x条件阻塞或挂起的进程，（有多个则选择其中一个）。如果没有，继续执行原进程，不产生任何后果。

3.4.5 管程

- 管程的条件变量
  - 管程x.signal的操作与信号量中的signal操作不同：
    - x.signal操作的作用，是重新启动一个被阻塞的进程，但如果没有被阻塞的进程，则x.signal操作不产生任何后果。
    - 而信号量中的signal总是要执行s=s+1操作，因而总是要改变信号量的状态。
  - 当进程P执行了x.signal操作唤醒进程Q后，怎样决定由哪个进程占用CPU执行，有两种处理方式：
    - P等待，直至Q离开管程或等待另一条件
    - Q等待，直至P离开管程或等待另一条件

3.4.5 管程

- 利用管程解决生产者-消费者问题
  - 建立一个管程，命名为Produer_Consumer，简称为PC。其中包括两个过程：
  - put(item)过程。生产者利用该过程将自己生产的产品投放到缓冲池中，并用整型变量count来表示在缓冲池中已有的产品数目，当count≥n时，表示缓冲池已满，生产者等待(阻塞)。
  - get(item)过程。消费者利用该过程从缓冲池中取出一个产品，当count≤0时，表示缓冲池已无可取的产品，消费者应等待（阻塞）。

3.4.5 管程

- 利用管程解决生产者-消费者问题
  - PC管程描述

```
type PC=monitor
  var in,out,count: integer;
  buffer: array[0..n-1] of item;
  notfull,notempty: condition;
  procedure entry put(item)
   begin
      if count≥n then notfull.wait; //生产者进程阻塞
      buffer[in] := nextp;
      in := (in+1) mod n;
      count := count+1;
      if notempty.queue then notempty.signal;
        //若等待队列非空，则唤醒队首的一个消费者进程
   end
```

```
procedure entry get(item)
    begin
       if count≤0 then notempty.wait;
       nextc := buffer[out];
       out := (out+1) mod n;
       count := count-1;
       if notfull.queue then notfull.signal;
      //若等待队列非空，则唤醒队首的一个生产者进程
   end
begin
  in :=out :=0; count :=0;  //初始化内部数据
end
```

3.4.5 管程

- 利用管程解决生产者–消费者问题
  - 生产者和消费者进程描述：

```
producer: begin
                repeat
                    produce an item in nextp;
                    PC.put(item);
                until false;
              end
consumer: begin
                repeat
                  PC.get(item);
                  consume the item in
nextc;
                until false;
              end
parbegin
producer;
consumer;
parend
```

3.5 进程通信

- 基本概念
  - 进程通信（Inter-Process Communication, IPC）是指两个或多个进程之间进行数据交互的过程。
  - 进程通信产生的原因：
    - 进程独立性：每个进程都拥有独立的内存地址空间，进程间的数据不能直接访问，保证了进程的安全性和独立性
    - 协同工作需求：实际应用中多个进程经常需要协同工作

3.5 进程通信

- **基本概念**
  - 进程通信的特点：
    - **多样性**：操作系统提供了多种进程通信机制，如管道、消息队列、共享内存、套接字等，以适应不同的应用场景。
    - **复杂性**：进程间通信需要考虑到数据的同步、互斥、缓冲等问题，以保证通信的正确性和效率

3.5 进程通信

- 进程通信的实现机制
  - 系统调用
    - 进程间通信通常需要借助操作系统的系统调用来实现。
  - 缓冲区
    - 为了提高通信效率，操作系统通常会为进程通信提供缓冲区。缓冲区可以暂时存储数据，减少进程间的直接交互，提高系统的并发性。
  - 同步机制
    - 为了保证进程间通信的正确性，操作系统提供了多种同步机制，如信号量、互斥锁、条件变量等。这些同步机制可以用于协调进程间的数据访问，避免数据竞争和死锁等问题。

3.5 进程通信

- 常见的进程通信方式
  - 管道
  - 消息队列
  - 共享内存
  - 套接字
  - 信号
  - 信号量

3.5 进程通信

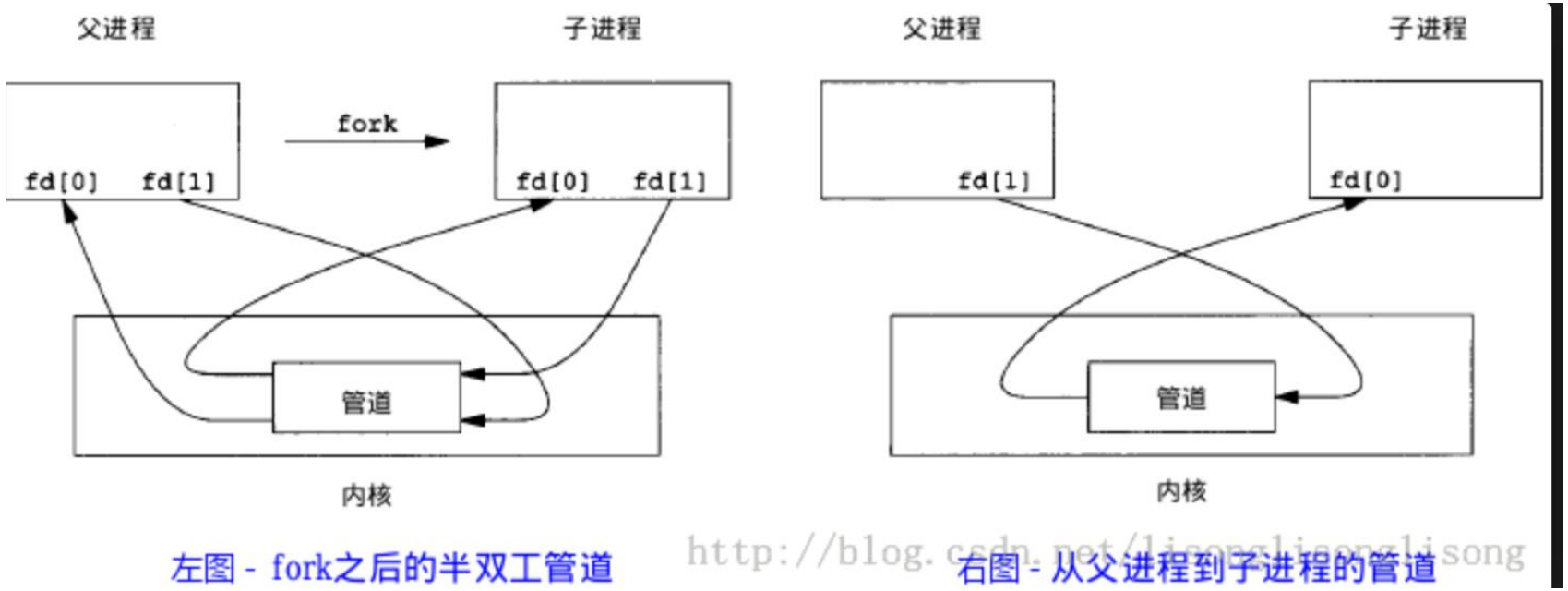
- 进程通信方式-管道(pipe)
  - 概念:
    - 管道是一种半双工的通信方式，用于连接一个读进程和一个写进程，实现数据的单向传输。
  - 分类:
    - 匿名管道：只能在父子进程之间使用，创建管道后，父进程和子进程各获得一个文件描述符，用于读写数据。
    - 命名管道（FIFO）：可以在不相关的进程之间使用，通过指定管道名称进行通信。

3.5 进程通信

- 进程通信方式-管道(pipe)
  - 特点：
    - 管道中的数据是顺序传输的，读操作会阻塞，直到有数据可读；写操作也会阻塞，直到有空间可写。
    - 从管道读数据是一次性操作，数据一旦被读走，它就从管道中被清除，释放空间以便写更多的数据。如果读取方进程需要多次读取相同的数据，写入方进程需要重新写入这些数据。

3.5 进程通信

- 进程通信方式-管道(pipe)



左图 - fork之后的半双工管道

http://blog.csdn.net/lisongli

右图 - 从父进程到子进程的管道

图3-9　无名管道通信

3.5 进程通信

- 进程通信方式-管道(pipe)

```c
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>

int main()
{
    int fd[2];//创建一个数组，这个数组有两个元素，fd[0]代表read,fd[1]代表write
    if(pipe(fd)==-1)
    {
        printf("管道创建失败\n");
        exit(1);
    }
    //管道创建成功:
    pid_t pid=fork();
    if(pid==-1)
    {
        printf("子进程产生失败\n");
        exit(1);
    }
    else if(pid==0)
    {
        close(fd[1]);//子进程关闭管道的写端
        char buff[128]={0};
        read(fd[0],buff,127);//子进程通过管道的读端向管道读取数据
        printf("子进程读到的数据：%s\n",buff);
        close(fd[0]);
    }
    else
    {
        close(fd[0]);//父进程关闭管道的读端
        char buff[128]={0};
        printf("父进程写入数据:");
        fgets(buff,128,stdin);
        write(fd[1],buff,strlen(buff));//父进程通过管道的写端向管道写入数据
        close(fd[1]);

    }
    exit(0);
}
```

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>

int main()
{
    int fr=open("fifo",O_RDONLY);
    if(fr==-1)//管道文件打开失败
    {
        printf("文件打开失败\n");
        exit(1);
    }
    else//管道文件打开成功，并输出管道文件的文件描述符
    {
        printf("fr=%d\n",fr);
    }

    while(1)
    {

        char buff[128]={0};//存放从管道文件中读取到的数据
        int n= read(fr,buff,127);//n用来存放read读到的字符个数
        if(n==0)//如果读到的字符是0个，就退出循环结束读操作
        {
            break;
        }
        printf("从管道中读到的数据：%s\n",buff);
    }
    close(fr);
    exit(0);
}
```

CSDN @

3.5 进程通信

- 进程通信方式-消息队列（Message Queue）
  - 概念：
    - 消息队列是一种将消息暂存于队列中的通信方式，支持异步通信
  - 特点：
    - 消息队列提供了先进先出（FIFO）的处理机制，保证消息的顺序性。进程可以向消息队列添加消息，也可以从消息队列读取消息
  - 应用：
    - 适用于需要异步通信的场景，如日志记录、任务分发等。

3.5 进程通信

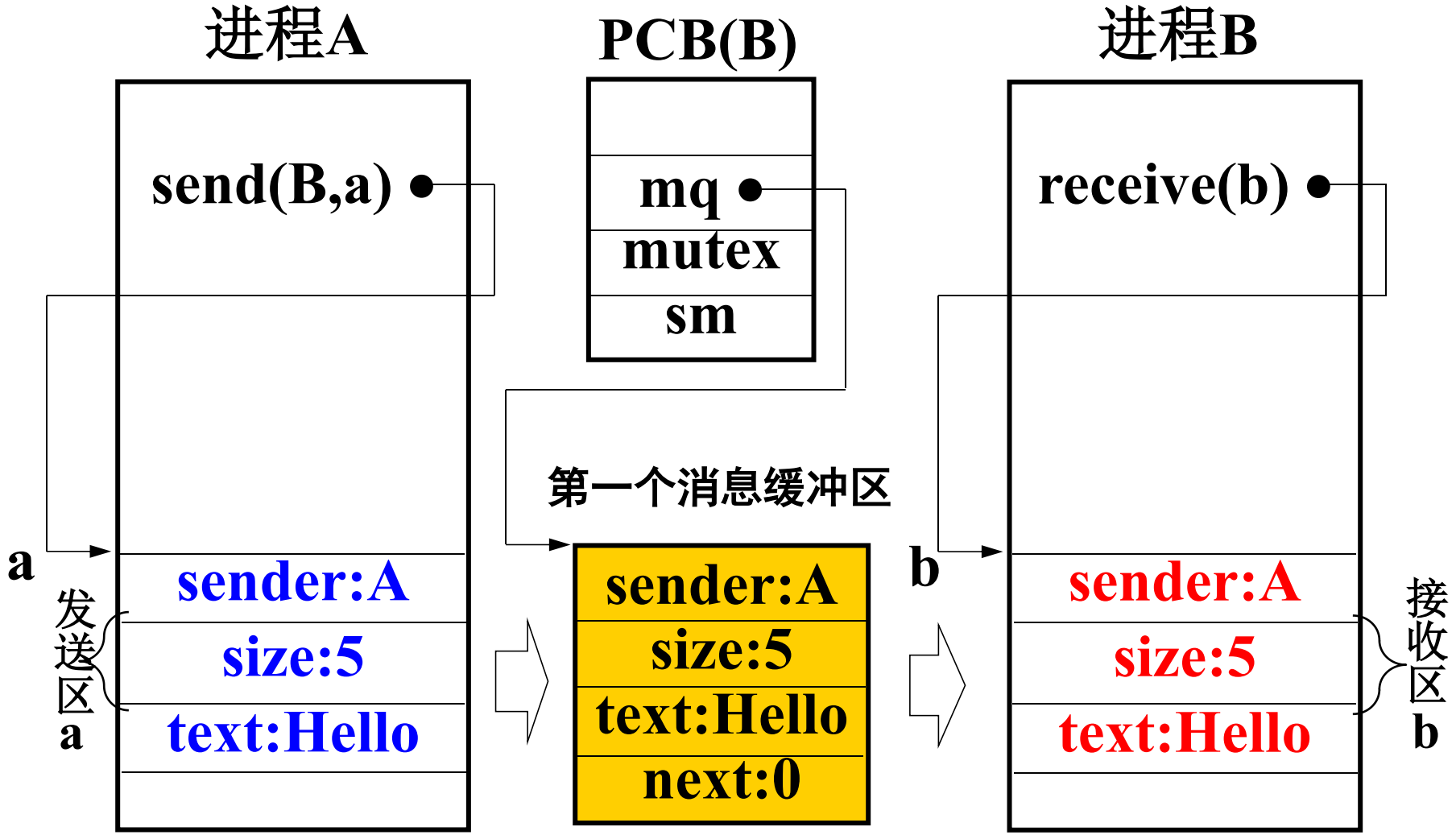


图3-10　消息队列通信

3.5 进程通信

- 进程通信方式-消息队列（Message Queue）

消息队列消息发送原语send（R,M）

```
begin
   在OS中分配M.size大小的缓冲区t;
   将M中的内容复制到t;
   得到进程R的PCB的指针q;
   wait(q.mutex);
     将t挂到队列q.mq队尾;
   signal (q.mutex);
   signal (q.sm);
end
```

消息队列消息接收原语Receive(N)

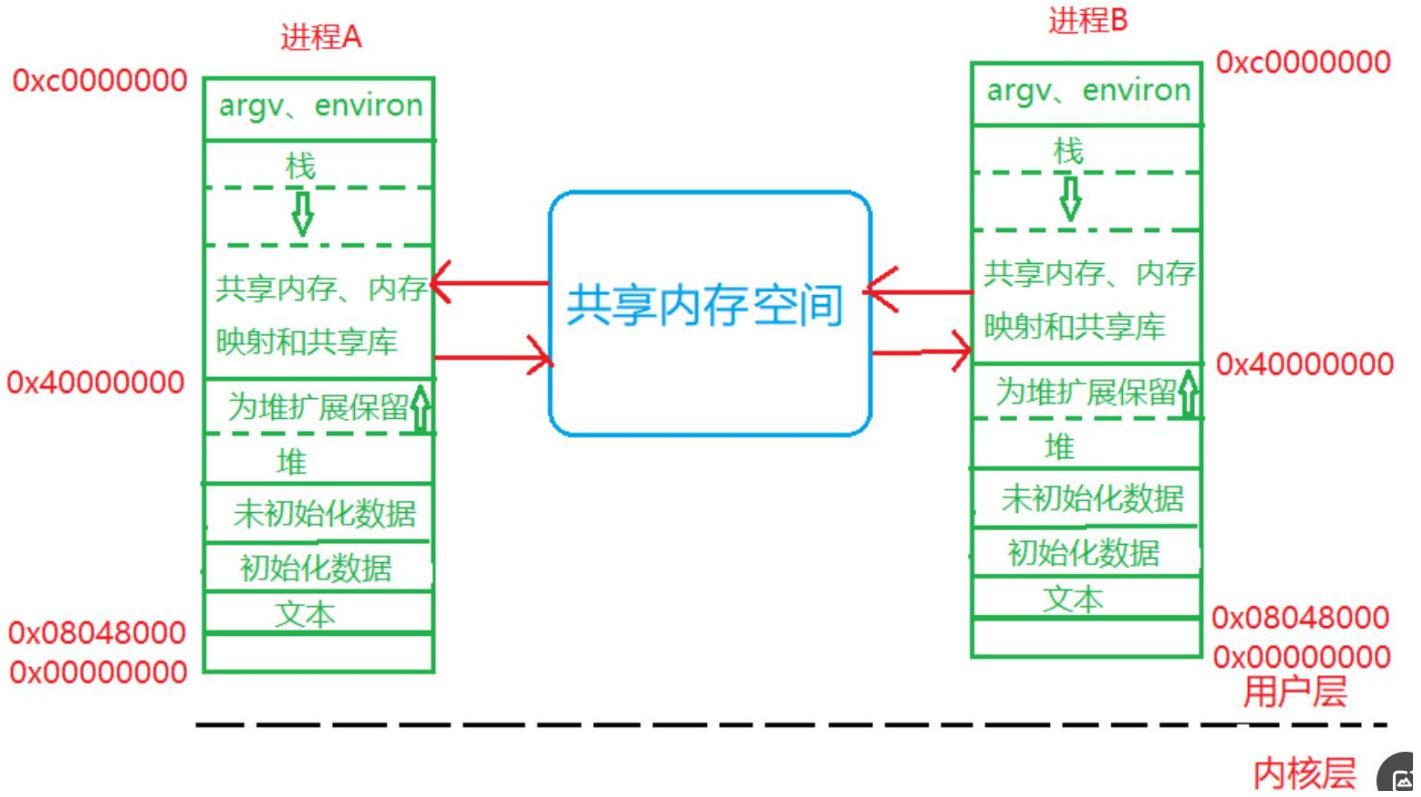
```
begin
   得到本进程PCB的指针q;
    wait(q.sm);
    wait(q.mutex);
      从q.mq队首取下一个缓冲区t;
    siganl(q.mutex);
    将t的内容复制到N，并释放t
end
```

3.5 进程通信

- 进程通信方式-共享内存（Shared Memory）
  - 概念：
    - 共享内存允许多个进程访问同一块内存空间，是最快的进程间通信方式。
  - 特点：
    - 信息交换速度快：进程可以直接读写共享内存区域来交换数据，不需要进行数据拷贝。
    - 需要使用同步机制（如信号量）来协调对共享内存的访问，避免数据竞争。
  - 应用：
    - 适用于需要频繁、大量数据交换的场景，如数据库服务器、实时系统等。

3.5 进程通信

- 进程通信方式-共享内存（Shared Memory）



3.5 进程通信

- 进程通信方式-套接字（Socket）
  - 概念:
    - 套接字是一种网络通信机制，允许进程在不同主机之间进行通信
  - 分类:
    - 流式套接字（TCP）：提供面向连接的、可靠的通信。
    - 数据报套接字（UDP）：提供面向无连接的、不可靠的通信。
  - 应用:
    - 适用于需要跨网络进行进程间通信的场景，如分布式系统、网络服务等。

3.5 进程通信

- 进程通信方式-套接字（Socket）

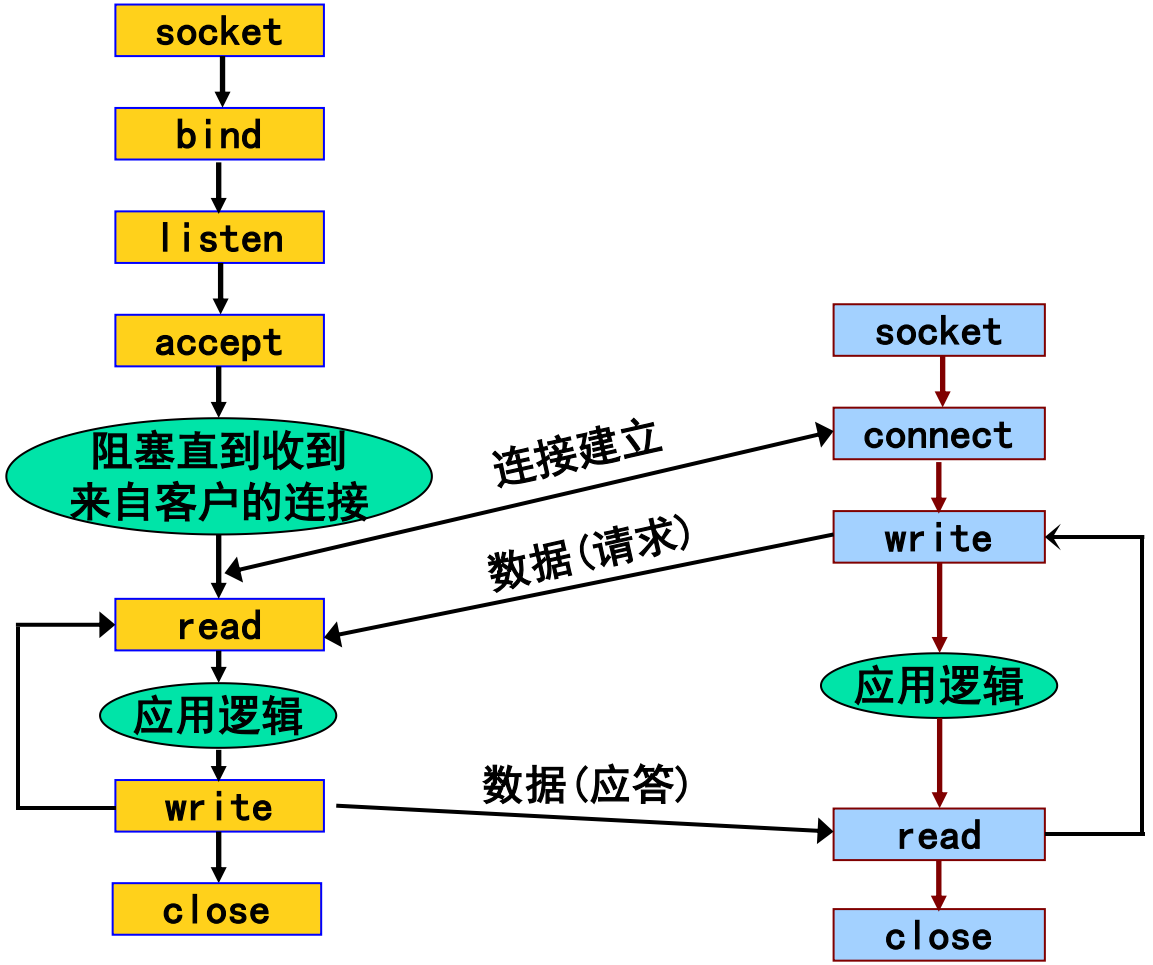


图3-11　流套接字通信模式

3.5 进程通信

- 进程通信方式-套接字（Socket）

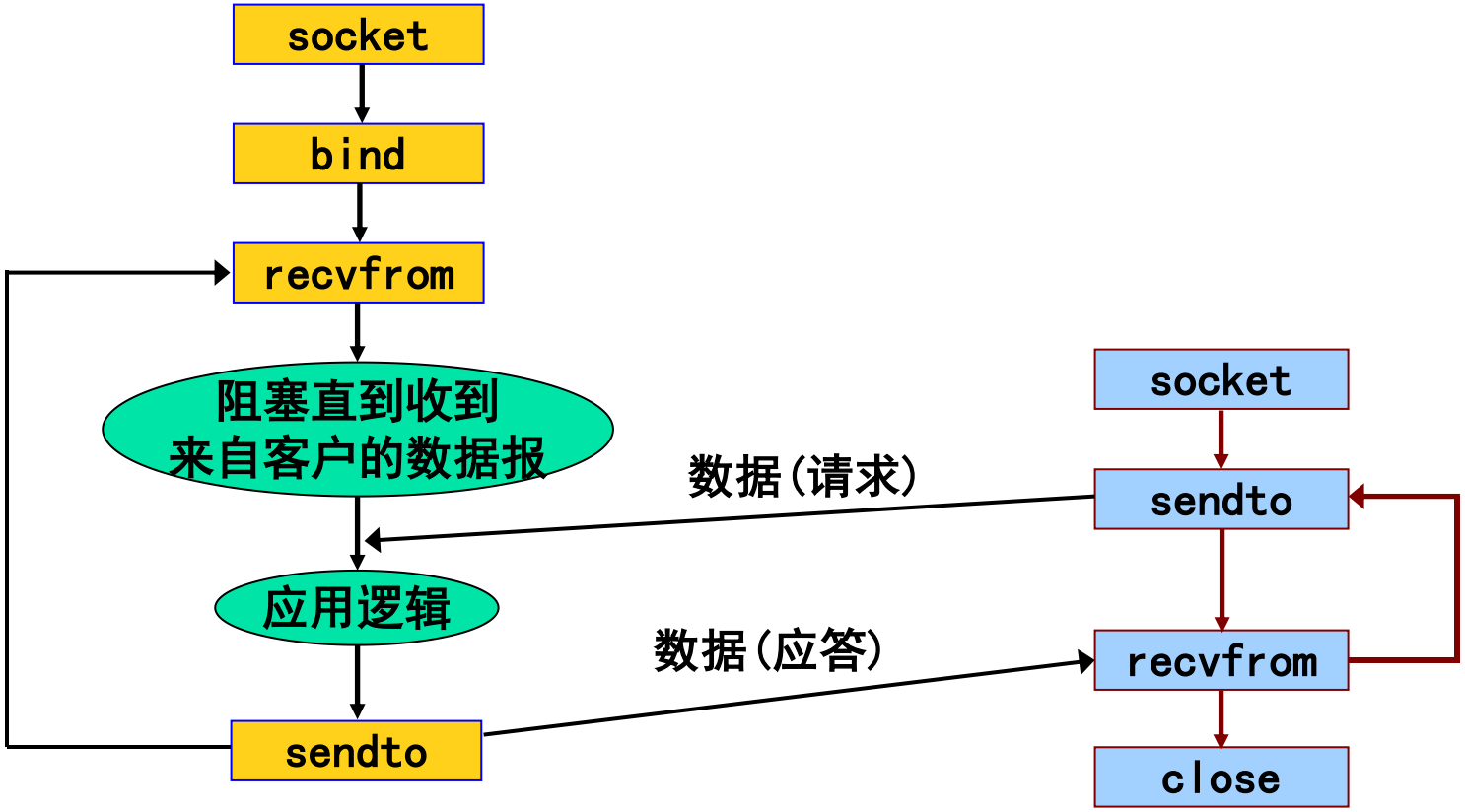


图3-12　数据报套接字通信模式

3.5 进程通信

- 进程通信方式-信号（Signal）
  - 概念：
    - 信号是一种异步通信机制，用于通知进程某个事件的发生。
  - 特点：
    - 信号通信简单快捷，但功能相对有限，只能传递有限的信息
    - 进程可以注册信号处理函数，在收到信号时执行相应的操作
  - 应用：
    - 适用于需要异步通知的场景，如中断处理、进程终止等。

3.5 进程通信

- 进程通信方式-信号量（Semaphore）
  - 概念：
    - 信号量是一种用于控制多个进程对共享资源访问的机制。
  - 特点：
    - 信号量通常用于实现进程的同步和互斥。通过P操作和V操作，信号量可以有效地管理共享资源的访问权限。
  - 应用：
    - 适用于需要同步和互斥访问共享资源的场景。

3.5 进程通信

实例：Linux系统进程通信函数

- int pipe(int fd[2]);
- int read(int fd, char *buf, int len);
- int write(int fd, char *buf, int len);
- FILE *popen(const char *command, const char *type);
- int pclose(FILE *stream);
- int mkfifo(const char *pathname, mode_t mode);
- int open(const char* filename,int flags[,mode_t mode]);
- int read(int fd, char *buf, int len);
- int write(int fd, char *buf, int len);
- int close (int fd);
- int msgget(key_t key, int oflag);
- int msgsnd(int msgid, const void *ptr, size_t length, int flag);
- ssize_t msgrcv(int msqid, void *ptr, size_t length, long type, int flag);
- int semget(key_t key, int nsems, int oflag);
- int semop(int semid, struct sembuf *opsptr, size_t nops);
- int shmget(key_t key, size_t size, int oflag);
- void * shmat(int shmid, const void *shmaddr, int flag);

3.6 线程

- 为什么引入线程
  - 进程的两个基本属性
    - 是拥有资源的基本单位
    - 是可被独立调度的基本单位
  - 进程并发执行时付出的时空开销
    - 由于进程拥有较多资源，使得在进程创建、进程撤销、切换时系统须付出较大的时空开销，限制了并发程度的进一步提高。
  - 为了减少进程切换和创建的开销，提高执行效率和节省资源，将进程独立性的两个属性分开，引入了线程(thread)。

3.6.1 线程的基本概念

- **线程概念**
  - 线程（Thread）是进程内的一个执行单位或进程内的一个可调度的实体，是CPU使用的基本单元。
  - 线程它由线程ID、程序计数器、寄存器集合和堆栈组成.
  - 一个进程可以创建一个或多个线程，属于同一进程的各线程共享其代码段、数据段和其他操作系统资源（如打开文件和信号）。
  - 在引入线程的现代操作系统中，进程是资源分配的单位，而线程是处理机调度的单位。

3.6.1 线程的基本概念

- **线程的优势**
  - 提高程序响应性
  - 资源共享
  - 并发执行
  - 轻量级
  - 提高程序吞吐量
  - 更好的利用系统资源

3.6.1 线程的基本概念

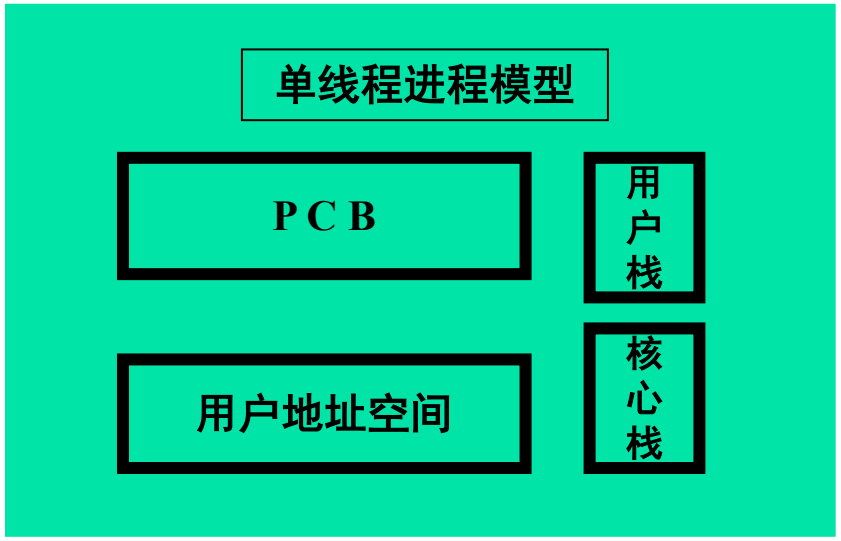
- 线程的适用范围
  - 多任务处理
  - 并行计算
  - 前后台处理/用户界面响应性
  - 后台服务
  - 资源共享与通信
  - 提高程序的可扩展性和可维护性
  - 实时系统

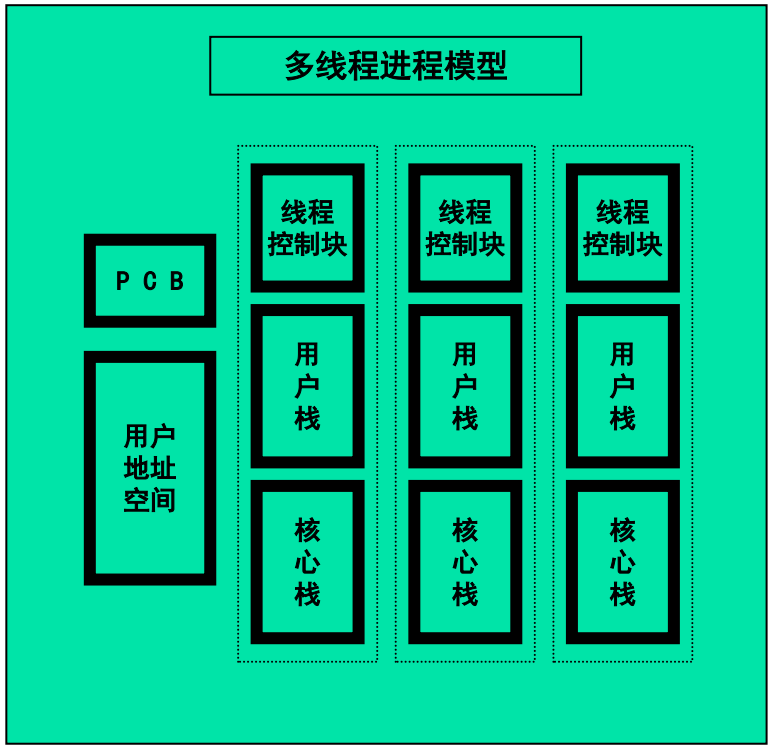
3.6.1 线程的基本概念

- 线程应用举例
  - 例1：LAN中的一个文件服务器，在一段时间内需要处理几个文件请求。有效的方法是：为每一个请求创建一个线程。
  - 例2：一个应用由几个独立部分组成，这几个部分不需要顺序执行，则每个部分可以以线程方式实现。当一个线程因I/O阻塞时，可以切换到同一应用的另一个线程。

3.6.1 线程的基本概念

- **单线程进程模型/**
  - **单线程进程模型**是指一个进程内只包含一个线程，该线程负责执行进程内的所有任务。
  - **多线程进程模型**是指一个进程内包含多个线程，每个线程可以并发执行不同的任务。





3.6.1 线程的基本概念

- 进程与线程的比较
  - 调度的基本单位：在支持线程的操作系统中，进程是资源分配的基本单位，而线程是处理机调度的基本单位。同一进程中，线程的切换不会引起进程的切换；从一个进程的线程切换到另一个进程中的线程，会引起进程的切换。
  - 并发性：同一进程内的线程可以并发执行，不同进程间的线程也可以并发执行，并发能力进一步增强。
  - 拥有资源：线程拥有自己的少量资源外，还允许多个线程共享该进程所拥有的资源。如，线程可以访问该进程地址空间中的每个虚地址，访问进程拥有的资源，如：已打开的文件，定时器，信号量，I/O设备等。

3.6.1 线程的基本概念

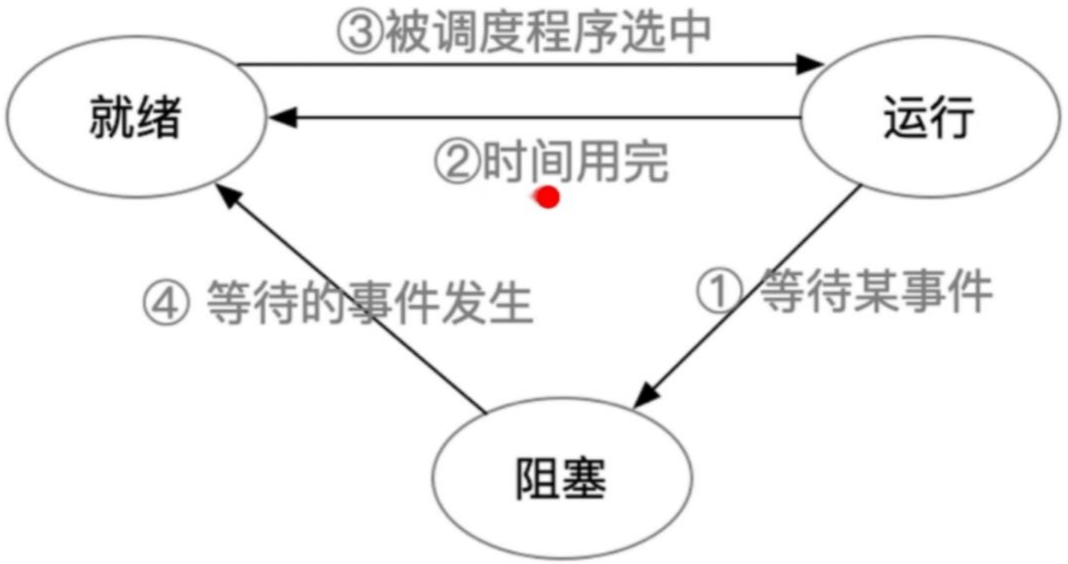
- 进程与线程的比较
  - 独立性：不同进程的地址空间是独立的，而同一进程内的线程共享同一地址空间。同一个进程中的不同线程之间的独立性要比不同进程之间的独立性低很多。一个进程的线程在另一个进程内是不可见的。
  - 系统开销：进程切换时将涉及到有关资源指针的保存以及地址空间的变化等问题；线程切换时，由于同一进程内的线程共享资源和地址空间，将不涉及资源信息的保存和地址变化问题，从而减少了操作系统的开销时间。
  - 支持多处理机系统：传统的进程，即单线程进程，该进程只能运行在一个处理机上；多线程进程，可以将一个进程的多个线程分配到多个处理机上并行运行。

3.6.1 线程的基本概念

- 线程的管理
  - 线程控制块（Thread Control Block，TCB）包括：
    - 线程标识符
    - 一组寄存器：程序计数器，状态寄存器，通用寄存器
    - 线程运行状态：就绪、运行、阻塞
    - 优先级：用于线程调度决策
    - 线程专有存储区：用于线程切换时保存现场信息
    - 堆栈指针：指向线程堆栈的顶部，用于过程调用时保存局部变量和返回地址
    - 信号屏蔽：指定线程需要屏蔽的信号集，用于线程的信号处理
    - 其他信息：线程调度所需的其他信息，指向线程所属进程的指针，以便线程可以访问进程的共享资源，线程名字等。

3.6.2 线程的管理

- **线程控制**
  - 与进程类似，线程也是一个动态的概念，有一个从创建到消亡的生命过程。在这一过程中它具多种状态，如运行、就绪、阻塞和终止等几个状态。



3.6.2 线程的管理

- 线程同步/互斥控制
  - 互斥锁（Mutex，Mutual Exclusion）
  - 信号量（Semaphore）
  - 条件变量（Condition Variable）
  - 读写锁（Read-Write Lock）
  - 屏障（Barrier）
  - 自旋锁（Spinlock）
  - 硬件支持的原子操作（Atomic Operations）

3.6.3 线程的实现

- 线程的分类
- 内核级线程（Kernel-Level Threads，KLT）
  - 是指由操作系统内核直接管理和调度的线程。
  - 线程的创建、撤销、调度和切换等操作都是在内核空间实现的，由操作系统内核完成的。
  - 内核为每个内核级线程维护一个线程控制块（Thread Control Block，TCB），通过TCB来感知和管理线程的存在。

3.6.3 线程的实现

- 线程的分类
- 内核级线程优势：
  - 支持真正的并行执行：多处理机系统中，内核能在不同处理器上可以同时调度多个线程
  - 高效的上下文切换：线程切换快，切换开销小
  - 线程阻塞处理：进程中的一个线程阻塞了，内核可以调度同一进程的另一线程以便执行
  - 内核任务支持：内核级线程通常用于执行与内核相关的任务，如中断处理、内存管理、调度等。这些任务需要较高的权限和直接访问硬件资源的能力，用内核级线程实现。
  - 系统资源访问：内核级线程可以直接访问硬件资源和内核数据结构，无需通过用户态和内核态之间的模式切换。

3.6.3 线程的实现

- 线程的分类

- 内核级线程局限：
  - 系统开销较大
    - 模式切换开销：内核级线程在用户态和内核态之间进行切换时，需要经历用户态→内核态→用户态的两次模式转换。
    - 线程管理开销：内核级线程的创建、销毁和调度等操作，都需要内核的参与。内核需要维护线程表、分配内存和处理器资源等。
  - 创建和销毁线程的速度较慢：内核级线程的创建和销毁需要内核的介入，涉及一系列复杂的操作。
  - 可移植差：内核级线程依赖于操作系统内核提供的线程机制。不同操作系统内核实现线程的方式、接口等存在差异。
  - 线程调度的灵活性受限：内核级线程的调度由操作系统内核统一进行，对于应用程序开发者来说，其对线程调度的干预能力相对有限。

3.6.3 线程的实现

- 线程的分类
- 用户级线程（User-Level Threads，ULT）
  - 是在用户空间实现的。
  - 对线程的创建、撤销、同步与通信等功能，都无需内核的支持，由应用程序利用线程库提供的函数来创建、管理和调度的线程。
  - 换句话说，内核完全不知道用户级线程的存在。

3.6.3 线程的实现

- 线程的分类
- 用户级线程优势：
  - 高效的线程切换：用户级线程的所有操作（如创建、切换和销毁）都在用户空间完成，不需要陷入内核态，因此避免了内核态和用户态之间的上下文切换开销。
  - 用户自定义调度：用户级线程允许每个进程定制自己的调度算法，线程管理比较灵活用户级线程的所有操作都在用户空间进行，无需切换到内核空间
  - 资源占用少：用户级线程不需要内核为每个线程分配独立的线程控制块和内核栈。
  - 易于实现：用户级线程可以通过线程库（如Java的线程库）来实现，线程的管理和操作更加方便和简单。
  - 用户级线程的实现与OS无关：对线程管理的代码属于用户程序的一部分，所有的应用程序都可以对之进行共享。

3.6.3 线程的实现

- 线程的分类
- 用户级线程局限：
  - 并发能力受限：当用户级线程中的一个线程执行系统调用或遇到阻塞操作时，操作系统会将整个进程阻塞。即使进程中的其他用户级线程是就绪状态，也无法继续执行。
  - 无法充分利用多处理器优势：在多处理器环境中，操作系统内核以进程为单位进行调度。每个进程中的用户级线程只能在单个处理器上分时复用，无法同时在不同的处理器上执行
  - 用户自定义调度：用户级线程需要用户程序自己实现线程的创建、调度、同步和通信等功能，需要程序员具备较高的并发编程能力。

3.6.3 线程的实现

- **线程的分类**
  - **组合式线程**指的是内核支持多个内核级线程的建立、调度、管理，同时也允许用户应用程序建立、调度和管理用户级线程。
  - 用户级线程与内核级线程的连接方式：**一对一**，**多对一**，**多对多**

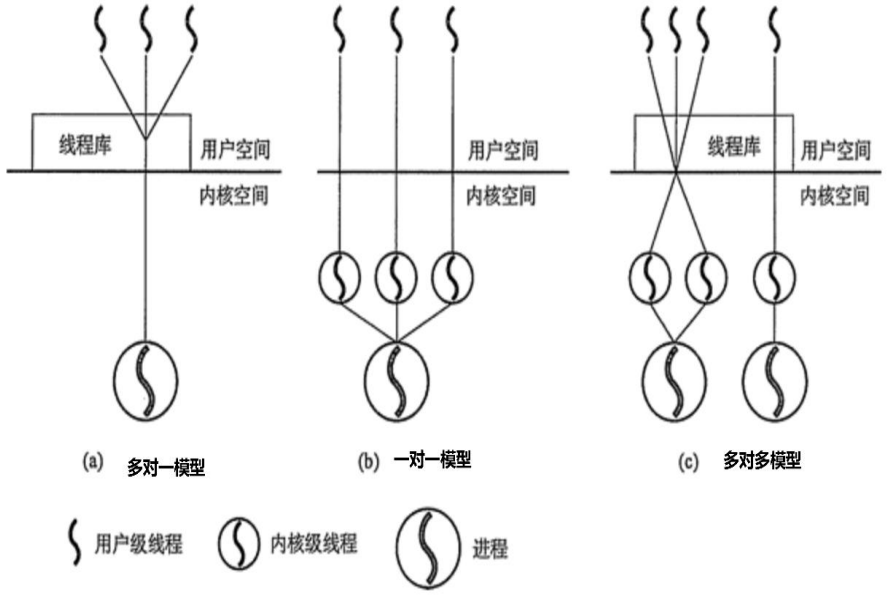


图3-14 多线程模型

3.6 线程的实现

- 线程的实现
  - 内核级线程的实现
    - 系统在创建一个新进程时，便为它分配一个任务数据区PTAD(Per Task Data Area)，其中包括若干个线程控制块TCB空间，PTAD保存在内核空间中。

3.6 线程

- 线程的实现
  - 用户级线程的实现
    - 1）运行时系统（runtime system）
      - 运行时系统是用于管理和控制用户级线程的函数（过程）的集合，包括线程创建、撤销、同步、通信、调度等线程管理函数。
      - 运行时系统中的所有函数都驻留在用户空间，并作为用户级线程与内核之间的接口。
      - 用户级线程在切换时不需要转入核心态，而是由运行时系统中的线程切换过程来执行切换任务。
      - 用户级线程需要系统资源时，将该要求传送给运行时系统，由后者通过相应的系统调用来获得系统资源。

3.6 线程

- 线程的实现
  - 用户级线程的实现
    - 2）轻量型进程LWP（Light Weight Process）
      - LWP可以共享进程所拥有的资源。
      - 每一个进程都可拥有多个LWP，每个LWP都有自己的数据结构（如TCB）。
      - LWP可通过系统调用来获得内核提供的服务，当一个用户级线程运行时，只要将它连接到一个LWP上，此时它便具有了内核提供支持线程的所有属性。

3.6 线程

- 线程的实现
  - 用户级线程的实现
    - 2）轻量型进程LWP（Light Weight Process）

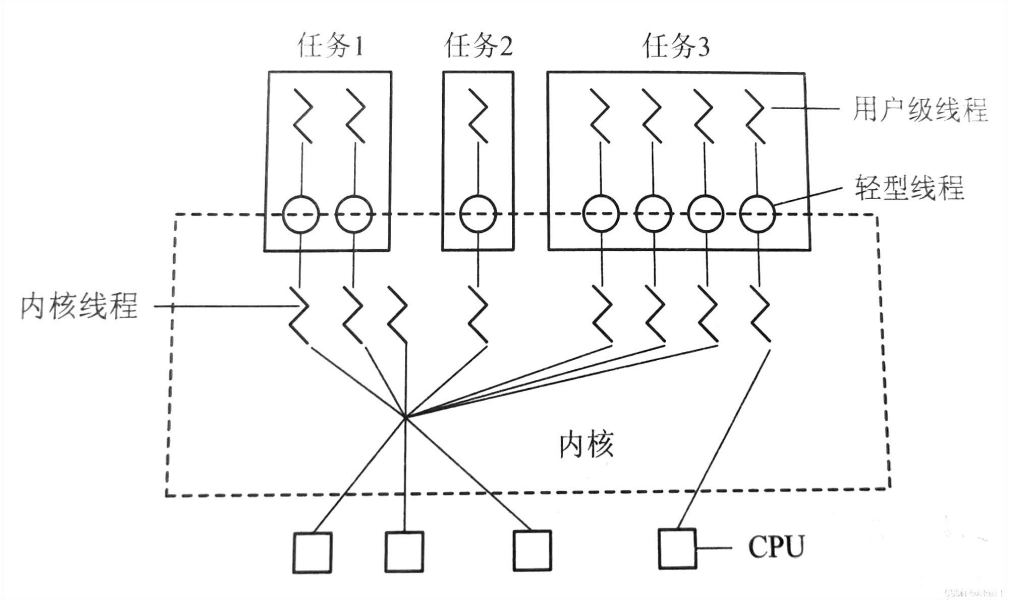


图3-15 LWP作为中间系统

3.6 线程

线程的实现

线程池

- 进程建立时创建若干LWP，把它们放在一个“池中”，称为线程池。
- 用户进程中的任一用户级线程都可以链接到LWP池中的任何LWP上，每个LWP连接到一个内核级线程上。
- 多个用户级线程可以多路复用一个LWP，但只有当前连接到LWP上的线程才能与内核通信。用户级线程不需要与内核通信时，并不需要LWP。
- 如果一个进程中含有多个LWP，则当一个LWP阻塞时，进程中的另一个LWP可以继续执行。
- 即使进程中的所有LWP全部阻塞，进程中的线程也能继续执行，只是不能再去访问内核。

3.6 线程

- 线程池
  - 线程池优势：
    - 降低资源开销：线程复用；资源控制
    - 提升响应速度：即时响应；并发处理
    - 提高线程的可管理性：统一管理；优化和监控
    - 避免资源泄露：自动回收，异常处理
    - 灵活的任务调度：任务队列；动态调整
    - 提高系统的稳定性和可靠性：控制并发度；隔离故障
    - 简化编程模型：简化代码，提高可读性